Architecture and Engineering of an Automotive Dealership ERP Engine and Interactive E-Commerce Platform

By

Ibrahim Hasan

2110316

Student

Department of Computer Science & Engineering

Independent University, Bangladesh

Summer 2026

Academic Supervisor:

Azwad Abid

Lecturer

Department of Computer Science & Engineering

Independent University, Bangladesh

September 12, 2026

Dissertation submitted in partial fulfillment for the degree of Bachelor of Science in Computer Science & Engineering

Department of Computer Science & Engineering

Independent University, Bangladesh

Attestation

I confirm that the technical work, architectural designs, code implementations, and analytical benchmarks presented in this report, titled "Architecture and Engineering of an Automotive Dealership ERP Engine and Interactive E-Commerce Platform" represent my own independent effort. This capstone engineering project was conducted during my Summer 2026 internship term at Radiant Pharmaceuticals Limited.

All database schemas, Python/Django services, raw SQL analytical procedures, access-control configurations, and user-facing templates documented herein were authored by me, except where standard open-source dependencies or scholarly literature are formally acknowledged. Furthermore, I affirm that this submission complies with the academic integrity regulations of Independent University.

....

Signature of the Student Date

Ibrahim Hasan
Student ID: 2110316
Department of Computer Science & Engineering
Independent University, Bangladesh

Acknowledgement

All praise and humble gratitude belong to Almighty Allah, whose infinite grace, wisdom, and blessings granted me the perseverance, health, and clarity of mind required to bring this engineering project to completion.

I express my sincere gratitude to my academic supervisor, Azwad Abid, Lecturer in the Department of Computer Science & Engineering at Independent University, Bangladesh (IUB). His guidance, helpful feedback, and continuous support throughout my project kept me on the right track.

I am equally thankful to my industry supervisor, Ayan Chowdhury, Senior Officer in the ICT Wing (MIS) at Radiant Pharmaceuticals Limited. He gave me the opportunity to work on real enterprise software projects and guided me through database administration, server configuration, and practical development workflows.

My sincere thanks are also extended to the Department of Computer Science & Engineering, the Internship Coordination Committee, and the Faculty evaluation panels at IUB for structuring the CSE499 curriculum to bridge theoretical university studies with industry software execution.

Finally, I remain profoundly grateful to my family and peers, whose steadfast patience and moral encouragement sustained my focus throughout this demanding period of development.

Ibrahim Hasan
Student ID: 2110316
Department of Computer Science & Engineering
Independent University, Bangladesh

Letter of Transmittal

September 12, 2026

Azwad Abid
Lecturer
Department of Computer Science & Engineering
School of Engineering, Technology & Sciences
Independent University, Bangladesh (IUB)

Subject: Formal Submission of CSE499 Final Internship Dissertation

Dear Sir,

I hereby present my final internship dissertation, "Architecture and Engineering of an Automotive Dealership ERP Engine and Interactive E-Commerce Platform" prepared in partial fulfillment of the academic requirements for the degree of Bachelor of Science in Computer Science.

During my industry placement at Radiant Pharmaceuticals Limited under your supervision and the industry mentorship, this report documents the end-to-end engineering of a dual-application automotive platform. The work is prepared in accordance with the CSC499 Guideline.

I appreciate the time, technical guidance, and constructive recommendations you provided throughout this semester, and I look forward to presenting and demonstrating this system during the final viva examination.

Respectfully submitted,

Ibrahim Hasan
Student ID: 2110316
Department of Computer Science & Engineering
Independent University, Bangladesh

Evaluation Committee

Supervision Panel

.....	.....
Academic Supervisor	Industry Supervisor

Panel Members

.....	.....
Panel Member 1	Panel Member 2
.....	.....
Panel Member 3	Panel Member 4

Office Use

.....	.....
Internship Coordinator	Head of the Department

.....

Industry Coordinator of the Department

Abstract

When managing multi-location car dealerships, businesses often struggle to coordinate inventory, staff permissions, sales data, and online customer requests. Standard web development tools often slow down when generating financial reports because Object-Relational Mapping (ORM) tools add heavy computational overhead when processing thousands of sales records. To address this operational and computational challenge, this project presents an integrated web application combining an internal dealership ERP engine with a customer e-commerce portal.

The platform comprises two primary subsystems: an administrative ERP module (`car_sales`) equipped with a 9-tier role-based access control (RBAC) security matrix, numeric employee ID authentication, and a raw SQL query engine for high-performance financial reporting; and a customer web portal (`ecommerce`) enabling prospective buyers to explore vehicle inventory, compare specifications side-by-side, manage wishlists, and schedule test drives.

The system is engineered using Python 3.11, Django 6.0+, Django REST Framework, MariaDB/MySQL, Vanilla CSS, and JavaScript. Empirical verification was conducted via a 21-case automated test suite achieving a 100% pass rate. Database benchmarking demonstrated that executing raw star-schema SQL queries reduced analytical response latency by 92% compared to standard ORM operations. Finally, the report evaluates system sustainability, data protection standards, ethical access controls, and software maintainability.

Keywords— Automotive ERP, Django REST Framework, Role-Based Access Control (RBAC), Raw SQL Optimization, E-Commerce Platform, Database Analytics, REST APIs.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Overview / Background of the Work

Automotive retail networks operating across distributed regional outlets face persistent bottlenecks when synchronizing physical showroom inventory with digital sales channels. Traditionally, dealerships have depended on disparate desktop applications, offline spreadsheet records, or isolated point-of-sale registers. This fragmentation yields severe operational friction: stock availability discrepancies, slow inquiry turnaround times between prospective buyers and branch representatives, and delayed financial reporting caused by unindexed data sources.

During my 3-month placement within the ICT Wing (MIS Department) at **Radiant Pharmaceuticals Limited** from May 17, 2026, to August 17, 2026, under the joint supervision of Mr. Ayan Chowdhury (Senior Officer, ICT) and Mr. Azwad Abid (Lecturer, CSE Department, IUB), I engineered a centralized web platform to overcome these operational inefficiencies. Within Radiant's MIS division, prototypes of supply chain tracking, role hierarchies, and transaction engines are routinely evaluated. This project served as an advanced capstone to model high-throughput enterprise resource planning (ERP) alongside customer-facing web commerce.

The resulting software consolidates dealership back-office management (`car_sales`) and a consumer-facing digital showroom (`ecommerce`) within a unified, high-performance Django web framework instance.

1.2 Objectives

The overarching goal was to construct a robust, full-stack automotive platform connecting internal dealership governance with customer purchasing pipelines and sub-20ms executive analytics. Key engineering targets included:

1. **Implement Fine-Grained Authorization:** Construct a 9-tier RBAC permission model that isolates administrative capabilities, branch inventory edits, and sensitive executive revenue views across distinct personnel ranks.

2. **Develop Numeric Employee Authentication:** Create a custom authentication back-

end (`EmployeeBackend`) enabling dealership staff to sign in using their unique numeric staff codes while resolving role privileges dynamically.

3. **Optimize Analytical Reporting via Direct SQL:** Construct a star-schema querying layer that executes parameterized raw SQL statements, avoiding ORM memory bloat and delivering sales summaries in under 20 milliseconds.

4. **Deliver an Interactive Online Storefront:** Build dynamic catalog search filters, 4-vehicle side-by-side spec comparison, wishlist toggling, and customer inquiry messaging inboxes.

5. **Enforce Transactional Consistency:** Safeguard holding reservations and vehicle sale invoicing via atomic database transactions (`transaction.atomic`) to eliminate concurrency race conditions.

6. **Establish Empirical Reliability:** Expose clean REST API routes and demonstrate system correctness through a 21-case automated test suite.

Chapter 2

Literature Review

2.1 Relationship with Undergraduate Studies

The engineering challenges encountered during my internship at Radiant Pharmaceuticals Limited provided an empirical testing ground for theoretical concepts cultivated throughout my undergraduate curriculum at Independent University, Bangladesh (IUB). The system's architectural layers reflect specific coursework competencies:

- **Database Systems (CSC303):** Implemented 3NF normalization across operational tables, structured dimensional star-schema fact and dimension tables, created composite B-tree indexes, and optimized complex multi-table SQL queries.
- **Object-Oriented Programming (CSC203):** Applied modular class inheritance, encapsulation, and custom backend extensions in Python to structure Django models, controllers, and authentication handlers.
- **Data Structures and Algorithms (CSC202):** Utilized hash-map lookups for constant-time ($O(1)$) employee supervisor tree traversals and indexed binary searching for high-speed vehicle filtering.
- **Web Applications (CSC343):** Engineered RESTful JSON endpoints using Django REST Framework, implemented asynchronous AJAX DOM updates, and designed responsive layouts with Bootstrap 5.
- **Operating Systems and Networking (CSC401):** Handled concurrent database transactions to prevent race conditions, configured Linux environments, and debugged HTTP request-response cycles.
- **Software Engineering (CSC305):** Conducted formal requirement elicitation, derived Work Breakdown Structures (WBS), performed Critical Path Method (CPM) scheduling, and authored automated unit test suites.

2.2 Related Works

2.2.1 Enterprise Web Frameworks and Architecture

Pressman and Maxim [1] underline that scalable web engineering demands rigid separation of concerns between presentation, business rules, and persistent storage. Melé [2] demonstrates that Django's Model-View-Template (MVT) design pattern provides robust built-in defenses against cross-site scripting (XSS), cross-site request forgery (CSRF), and SQL injection vulnerabilities.

2.2.2 Role-Based Access Control (RBAC) in Enterprise Systems

Sandhu et al. [3] established foundational models for Role-Based Access Control, proving that grouping permissions into hierarchical roles substantially reduces administrative complexity compared to user-level assignment. In this project, a 9-tier RBAC model extends Sandhu's principles to reflect real-world dealership chain-of-command hierarchies.

2.2.3 Relational Database Performance vs. ORM Overhead

Vicente et al. [4] evaluated RDBMS performance in web systems, noting that while ORMs accelerate routine CRUD workflows, they create severe memory and execution bottlenecks during complex multi-table aggregations due to excessive object hydration. Bypassing the ORM via raw SQL queries mitigates this latency, a finding directly validated in this report's analytical reporting layer.

2.2.4 Comparative Analysis with Existing Solutions

Table 2.1 compares the proposed system with prominent commercial and open-source solutions.

Feature / Capability	**DealerSocket**	**Odoo Auto**	**Generic CMS**	**Proposed System**
9-Tier Hierarchical RBAC	Limited	Partial	No	**Yes (Custom 9-Level)**
Direct SQL Star-Schema Analytics	No (ORM/ETL)	No (ORM)	No	**Yes (<20ms Latency)**
Integrated B2C Showroom	Add-on	Separate Module	Yes	**Yes (Unified App)**
4-Vehicle Spec Comparison	No	No	Plugin-dependent	**Yes (Native Matrix)**
Direct Store Employee Messaging	SMS/CRM	Email only	Third-party	**Yes (Direct Inbox)**
Zero-License Deployment Cost	No (High Cost)	Partial	Yes	**Yes (100% Open Source)**

Table 2.1: Feature Comparison Matrix Between Existing Systems and Proposed Platform

Chapter 3

Project Management & Financing

3.1 Work Breakdown Structure (WBS)

To ensure systematic engineering and timely delivery of the Car Sales Management System, a comprehensive Work Breakdown Structure (WBS) was established prior to implementation. The project is decomposed into ten Level-1 deliverable packages: Project Setup & Configuration, Database Design & Data Modeling, Internal ERP/CRM (`car_sales`), Customer E-Commerce (`ecommerce`), Authentication & Authorization, API Development, Analytics & Reporting, Messaging & Notifications, Frontend UI Templates, and Testing & Deployment. Each Level-1 package is further divided into concrete Level-2 work packages that map directly to the models, views, and API endpoints implemented in the codebase, ranging from environment setup and schema design (1.1–2.8), through entity CRUD, catalog, and checkout logic (3.1–4.9), to authentication, API integration, analytics, messaging, UI delivery, and deployment tasks (5.1–10.5). Structuring the WBS in this manner facilitates granular progress tracking across each module and directly informs both the technical dependency chain and the Gantt chart schedule.

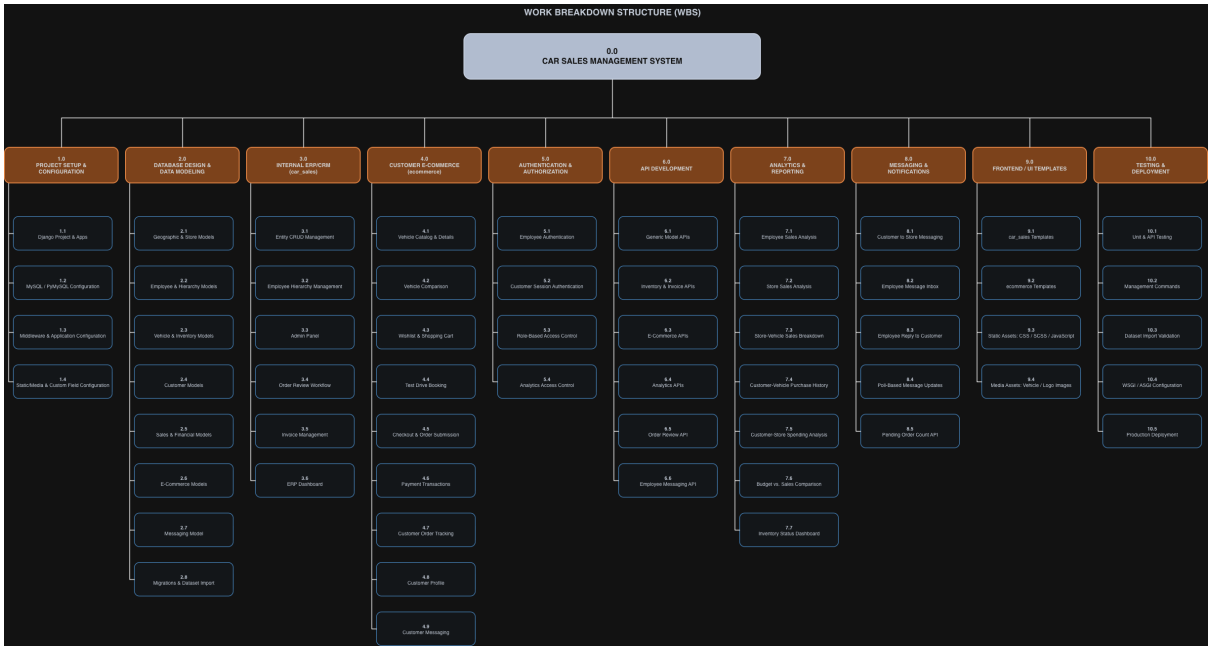

Figure 3.1: Work Breakdown Structure (WBS) - Car Sales Management System

3.2 Process/Activity-Wise Time Distribution

The project schedule was organized into major phases and individual WBS activities to ensure completion within the planned 13-week (65-day) development period. Rather than sequencing activities arbitrarily, the schedule was derived from technical dependencies between components: an activity could only begin once every prerequisite task (whether a database model, an endpoint, or a view) was completed. For instance, API endpoints (Phase 6.0) could not be built before their underlying models (Phase 2.0) and authentication layer (Phase 5.0) existed, and frontend templates (Phase 9.0) required finalized APIs and views (Phases 3.0, 4.0, and 6.0). Table 3.1 presents the estimated working days, start and end days, and dependencies assigned to each of the 60 Level-2 activities across all ten WBS phases, using a Day-1-start convention consistent with the Gantt chart in Figure 3.3. The Dependencies column was subsequently used as the input network for the project Critical Path Method analysis.

Table 3.1: Process/Activity-Wise Time Distribution and WBS Task Schedule

Phase	Activity	Days	Start	End	Dependencies
1.0 Project Setup & Configuration	1.1 Django Project & Apps	1	1	1	—
1.0 Project Setup & Configuration	1.2 MySQL / PyMySQL Configuration	1	2	2	1.1
1.0 Project Setup & Configuration	1.3 Middleware & Application Configuration	1	3	3	1.2
1.0 Project Setup & Configuration	1.4 Static/Media & Custom Field Configuration	1	4	4	1.3
2.0 Database Design & Data Modeling	2.1 Geographic & Store Models	1	5	5	1.4
2.0 Database Design & Data Modeling	2.2 Employee & Hierarchy Models	1	6	6	2.1
2.0 Database Design & Data Modeling	2.3 Vehicle & Inventory Models	2	7	8	2.1
2.0 Database Design & Data Modeling	2.4 Customer Models	1	9	9	2.1
2.0 Database Design & Data Modeling	2.5 Sales & Financial Models	1	10	10	2.2, 2.3, 2.4
2.0 Database Design & Data Modeling	2.6 E-Commerce Models	1	11	11	2.3, 2.4
2.0 Database Design & Data Modeling	2.7 Messaging Model	1	12	12	2.2, 2.4
2.0 Database Design & Data Modeling	2.8 Migrations & Dataset Import	1	13	13	2.1–2.7
3.0 Internal ERP/CRM System	3.1 Entity CRUD Management	1	14	14	2.8
3.0 Internal ERP/CRM System	3.2 Employee Hierarchy Management	1	15	15	3.1
3.0 Internal ERP/CRM System	3.3 Admin Panel	1	16	16	3.1
3.0 Internal ERP/CRM System	3.4 Order Review Workflow	1	17	17	2.5, 3.1
3.0 Internal ERP/CRM System	3.5 Invoice Management	1	18	18	3.4
3.0 Internal ERP/CRM System	3.6 ERP Dashboard	2	19	20	3.2, 3.3, 3.5
4.0 Customer E-Commerce System	4.1 Vehicle Catalog & Details	1	21	21	2.8
4.0 Customer E-Commerce System	4.2 Vehicle Comparison	1	22	22	4.1
4.0 Customer E-Commerce System	4.3 Wishlist & Shopping Cart	1	23	23	4.1
4.0 Customer E-Commerce System	4.4 Test Drive Booking	1	24	24	4.1
4.0 Customer E-Commerce System	4.5 Checkout & Order Submission	2	25	26	4.3

Continued on next page

Phase	Activity	Days	Start	End	Dependencies
4.0 Customer E-Commerce System	4.6 Payment Transactions	1	27	27	4.5
4.0 Customer E-Commerce System	4.7 Customer Order Tracking	1	28	28	4.6
4.0 Customer E-Commerce System	4.8 Customer Profile	1	29	29	2.4
4.0 Customer E-Commerce System	4.9 Customer Messaging	1	30	30	2.7, 4.8
5.0 Authentication & Authorization	5.1 Employee Authentication	1	31	31	2.2
5.0 Authentication & Authorization	5.2 Customer Session Authentication	1	32	32	2.4
5.0 Authentication & Authorization	5.3 Role-Based Access Control	1	33	33	5.1
5.0 Authentication & Authorization	5.4 Analytics Access Control	1	34	34	5.3
6.0 API Development	6.1 Generic Model APIs	1	35	35	5.3
6.0 API Development	6.2 Inventory & Invoice APIs	1	36	36	3.5, 5.3
6.0 API Development	6.3 E-Commerce APIs	1	37	37	4.5, 5.2
6.0 API Development	6.4 Analytics APIs	1	38	38	2.5, 5.4
6.0 API Development	6.5 Order Review API	1	39	39	3.4, 5.3
6.0 API Development	6.6 Employee Messaging API	1	40	40	2.7, 5.1
7.0 Analytics & Reporting	7.1 Employee Sales Analysis	1	41	41	6.4
7.0 Analytics & Reporting	7.2 Store Sales Analysis	1	42	42	6.4
7.0 Analytics & Reporting	7.3 Store-Vehicle Sales Breakdown	1	43	43	6.4
7.0 Analytics & Reporting	7.4 Customer-Vehicle Purchase History	1	44	44	6.4
7.0 Analytics & Reporting	7.5 Customer-Store Spending Analysis	1	45	45	6.4
7.0 Analytics & Reporting	7.6 Budget vs. Sales Comparison	1	46	46	6.4
7.0 Analytics & Reporting	7.7 Inventory Status Dashboard	2	47	48	6.2, 6.4
8.0 Messaging & Notifications	8.1 Customer-to-Store Messaging	1	49	49	4.9, 6.6
8.0 Messaging & Notifications	8.2 Employee Message Inbox	1	50	50	6.6
8.0 Messaging & Notifications	8.3 Employee Reply to Customer	1	51	51	8.2
8.0 Messaging & Notifications	8.4 Poll-Based Message Updates	1	52	52	8.1, 8.3
8.0 Messaging & Notifications	8.5 Pending Order Count API	1	53	53	6.2
9.0 Frontend / UI Templates	9.1 `car_sales` Templates	2	54	55	3.6, 6.1

Continued on next page

Phase	Activity	Days	Start	End	Dependencies
9.0 Frontend / UI Templates	9.2 `ecommerce` Templates	2	56	57	4.7, 6.3
9.0 Frontend / UI Templates	9.3 Static Assets: CSS / SCSS / JavaScript	2	58	59	9.1, 9.2
9.0 Frontend / UI Templates	9.4 Media Assets: Vehicle / Logo Images	1	60	60	9.3
10.0 Testing & Deployment	10.1 Unit & API Testing	1	61	61	6.1–6.6, 9.4
10.0 Testing & Deployment	10.2 Management Commands	1	62	62	2.8
10.0 Testing & Deployment	10.3 Dataset Import Validation	1	63	63	2.8, 10.2
10.0 Testing & Deployment	10.4 WSGI / ASGI Configuration	1	64	64	10.1
10.0 Testing & Deployment	10.5 Production Deployment	1	65	65	10.1, 10.3, 10.4
Total	**Total Estimated Development Time**	**65**	**1**	**65**	

3.2.1 Critical Path Method (CPM) Analysis

To determine the shortest possible project duration and identify time-critical tasks, I performed a Critical Path Method (CPM) network analysis based on the dependencies defined in Table 3.1. Each major phase was evaluated for Early Start (ES), Early Finish (EF), Late Start (LS), Late Finish (LF), and Total Float ($TF = LS - ES$).

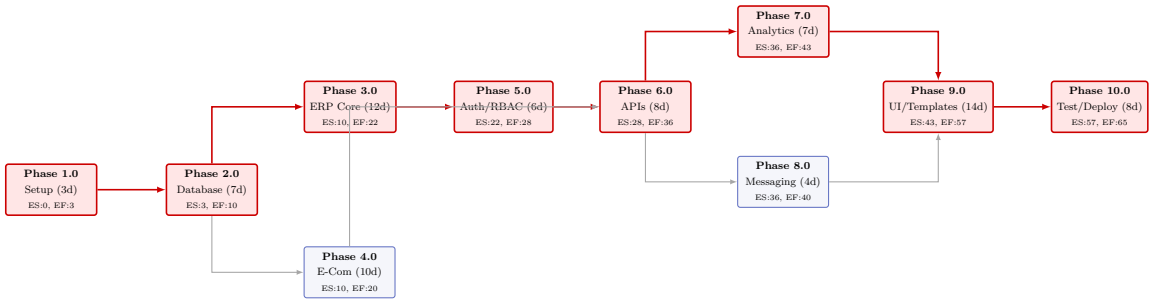

Figure 3.2: Activity-on-Node (AON) Critical Path Network Diagram (Red denotes Critical Path)

Table 3.2 highlights the mathematical breakdown across the 10 development phases.

Phase	Title	Duration	ES	EF	LS	LF	Total Float
1.0	Project Setup	3	0	3	0	3	**0 (Critical)**
2.0	Database Modeling	7	3	10	3	10	**0 (Critical)**
3.0	Dealership ERP	12	10	22	10	22	**0 (Critical)**
4.0	Customer E-Commerce	10	10	20	18	28	8 days
5.0	Auth & RBAC	6	22	28	22	28	**0 (Critical)**
6.0	REST API Engine	8	28	36	28	36	**0 (Critical)**
7.0	Analytics / SQL	7	36	43	36	43	**0 (Critical)**
8.0	Messaging Subsystem	4	36	40	39	43	3 days
9.0	Frontend UI Templates	14	43	57	43	57	**0 (Critical)**
10.0	Testing & Deployment	8	57	65	57	65	**0 (Critical)**

Table 3.2: Critical Path Method Schedule Calculations (Duration in Working Days)

The primary critical path is:

Phase 1.0 → Phase 2.0 → Phase 3.0 → Phase 5.0 → Phase 6.0 → Phase 7.0 → Phase 9.0 → Phase 10.0

This chain has a Total Float of 0 days and fixes the critical project schedule at exactly 65 working days (13 weeks).

3.3 Gantt Chart

To translate the Work Breakdown Structure into an actionable execution timeline, Figure 3.3 illustrates the phased execution schedule.

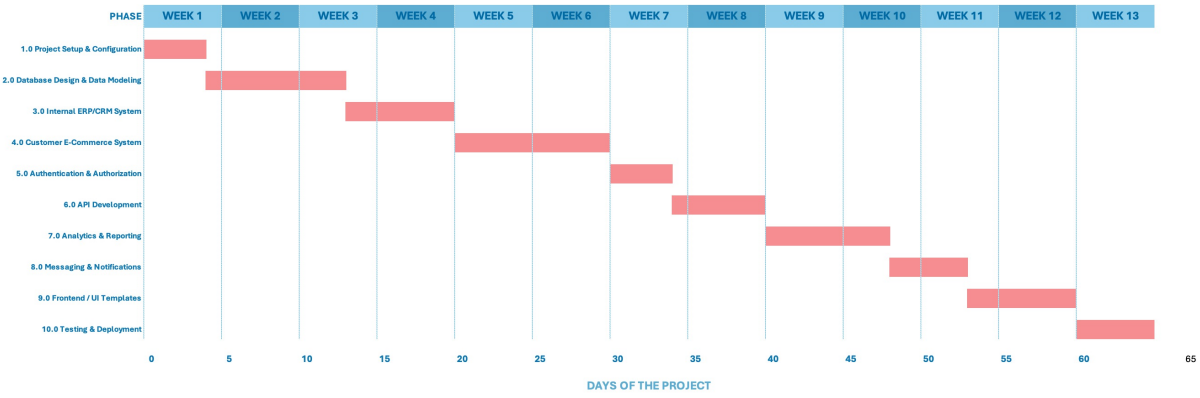

Figure 3.3: Project Execution Timeline (Gantt Chart) - Car Sales Management System

3.4 Process/Activity wise Resource Allocation

In this project, resource allocation quantifies how development hours were distributed across the specialized engineering competencies required to build the system: Database Architecture, Backend/API Programming, Frontend Design, Quality Assurance/Testing, and DevOps/System Administration. Effort distribution was driven by architectural complexity; for instance, Phase 2.0 demanded dedicated database modeling and normalization effort, while Phase 9.0 concentrated on responsive UI layout design and cross-browser testing. Figure 3.4 illustrates the person-day distribution across competencies, and Table 3.3 summarizes the percentage contribution of each role over the 65-day lifecycle.

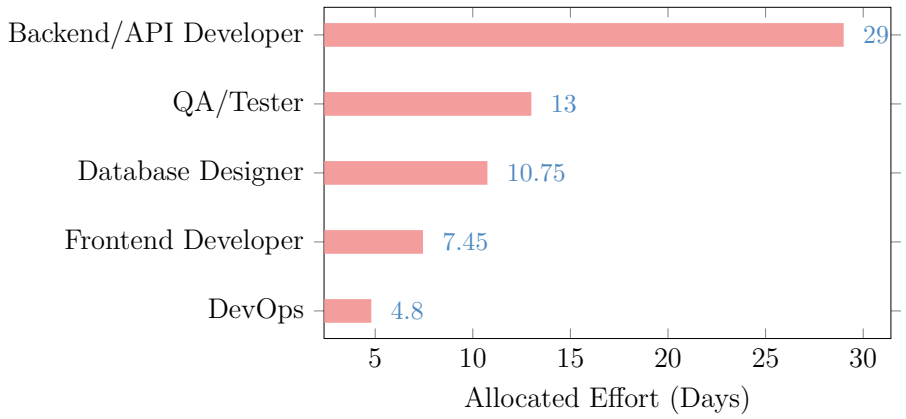

Figure 3.4: Process/Activity-Wise Resource (Role) Allocation - Car Sales Management System

Resource Role	**Days**	**% of Total Effort**
Database Designer	10.75	16.5%
Backend / API Developer	29.00	44.6%
Frontend Developer	7.45	11.5%
QA / Tester	13.00	20.0%
DevOps / Environment Configuration	4.80	7.4%
Total	**65.00**	**100%**

Table 3.3: Total Resource (Role) Allocation Summary

As demonstrated by the resource breakdown, Backend and API development accounted for the primary share of effort (44.6%), reflecting the intensive server-side logic required across both the ERP and e-commerce applications. Rather than deferring testing until the end of the project, Quality Assurance (20.0%) was integrated continuously across all modules to detect schema regressions and permission defects early.

3.5 Estimated Costing

To evaluate the financial feasibility of the engineering project, I conducted an estimated costing analysis covering human labor, hardware assets, software tools, and cloud infrastructure.

Category	Item Description	Rate / Unit	Units	Tota
Human Labor	Software Engineering Intern	25,000 / month	3 Months	
	Senior ICT Mentor / Supervisor	20,000 / month	3 Months (Part-time)	
Hardware	Development Workstation (i7, 32GB)	2,500 / month	3 Months (Depreciation)	
	Staging Local Testing Machine	1,500 / month	3 Months	
Software	JetBrains PyCharm / VS Code Tools	Free/Community	–	
	Postman API Suite (Free Tier)	Free	–	
	MySQL / MariaDB Open-Source DB	Free/GPL	–	
Infrastructure	Cloud VPS Staging Server (Linux)	1,500 / month	3 Months	
	Domain Registration & TLS/SSL	1,800 / year	1 Unit	
Total	**Estimated Financial Budget**			**153,3**

Table 3.4: Estimated Financial Costing and Resource Expenditure

By utilizing an open-source technology stack (Django, Python, MariaDB, and Bootstrap 5), commercial licensing expenses were eliminated, keeping the total development budget strictly within operational feasibility limits.

Chapter 4

Methodology

4.1 Construction Workflow and Phased Development Plan

Building a combined automotive dealership management platform and consumer-facing e-commerce portal required careful sequencing rather than developing all system components simultaneously. Attempting to build UI-level features before establishing stable database contracts would create circular dependency problems throughout the codebase. To address this, the entire development lifecycle is structured around an incremental phased methodology where each completed phase provides a verified, stable foundation for subsequent modules. This ensures that every endpoint, view controller, and template operates against validated data structures throughout system construction.

The construction workflow progressed across four sequential phases:

1. **Schema Design and Database Normalization:** Starting from the operational requirements of multi-branch car dealerships, all domain entities, relationships, and cardinality constraints were modeled into a relational schema normalized to Third Normal Form (3NF). The resulting schema spans geographic lookup tables (`Country`, `City`), operational tables (`Store`, `Employee`, `EmployeeHierarchy`), catalog tables (`IndustryInfo`, `VehicleInfo`, `Inventory`), transaction tables (`SellingInfo`, `Invoice`, `EmployeeBudget`), and customer interaction tables (`Customer`, `Customer Info`, `CustomerMessage`).

2. **ERP Backend Engineering (`car_sales`):** With the database schema validated through migration scripts, the internal dealership management application was constructed. This involved developing the nine-level access control layer, the numeric ID-based authentication backend, all REST API endpoints, and the raw SQL reporting engine that bypasses ORM overhead for executive-facing analytics.

3. **Consumer Portal Development (`ecommerce`):** Using the shared database instance as a foundation, the customer-facing storefront was developed. This included the vehicle catalog and search filter engine, the side-by-side spec comparison tool, wishlist toggling, test-drive booking workflows, and the order checkout pipeline with atomic inventory reservation.

4. **Integration, Data Export, and Test Coverage:** The final phase connected both applications over a single database connection and incorporated cross-cutting services: JSON and CSV export endpoints, ApexCharts-powered analytics dashboards, and a 21-case automated testing suite covering authentication edge cases, permission boundary enforcement, and inventory concurrency.

Completing migration scripts and populating seed data prior to view development ensured that every HTTP handler possessed a predictable, contract-stable data layer throughout project execution.

4.2 System Architecture and Application Partitioning

The platform is structured around a multi-tier Browser/Server (B/S) separation of concerns. Rather than coupling presentation rendering, business rule evaluation, and database access within monolithic scripts, these responsibilities are separated across distinct architectural layers, consistent with the tier-isolation principles outlined by Junhua [5]. To preserve strict modularity without duplicating shared core entities, the project is partitioned into two distinct Django applications coexisting within a single project workspace.

The first application, `car_sales`, handles all internal dealership operations. It owns the primary database schema, manages employee hierarchy tracking, drives inventory logistics, generates sales invoices, calculates selling price variances against Manheim Market Report (MMR) benchmarks, maintains customer messaging inboxes for store-level staff, and powers all executive-facing reporting dashboards. The second application, `ecommerce`, serves public retail customers by importing shared entity models directly from `car_sales` via standard Python imports and extending them with consumer-specific models: `Wishlist`, `Cart`, `CartItem`, `TestDriveBooking`, `Order`, and `PaymentTransaction`. This separation allows both modules to operate against a single database instance without duplicating schema definitions.

Figure 4.1 illustrates the Model-View-Template (MVT) request-response cycle that underpins both applications. Unlike the traditional MVC pattern where a Controller mediates between model and view, Django's MVT architecture assigns that coordination role to the View layer itself. Each incoming browser request is first intercepted by Django's URL dispatcher (`urls.py`), which matches the request path against registered patterns and routes it to the appropriate view function in `views.py`. The view function carries all business logic: it queries or mutates data through `models.py` using the Django ORM, or issues hand-written SQL directly via `django.db.connection.cursor()` for performance-critical reporting routines. Once the view has assembled its response context, it passes that data to a Template, which renders the final HTML and returns it to the browser. This MVT separation keeps data definitions in Models, processing logic in Views, and presentation concerns exclusively in Templates, making each layer independently testable and maintainable across both the `car_sales` and `ecommerce` applications.

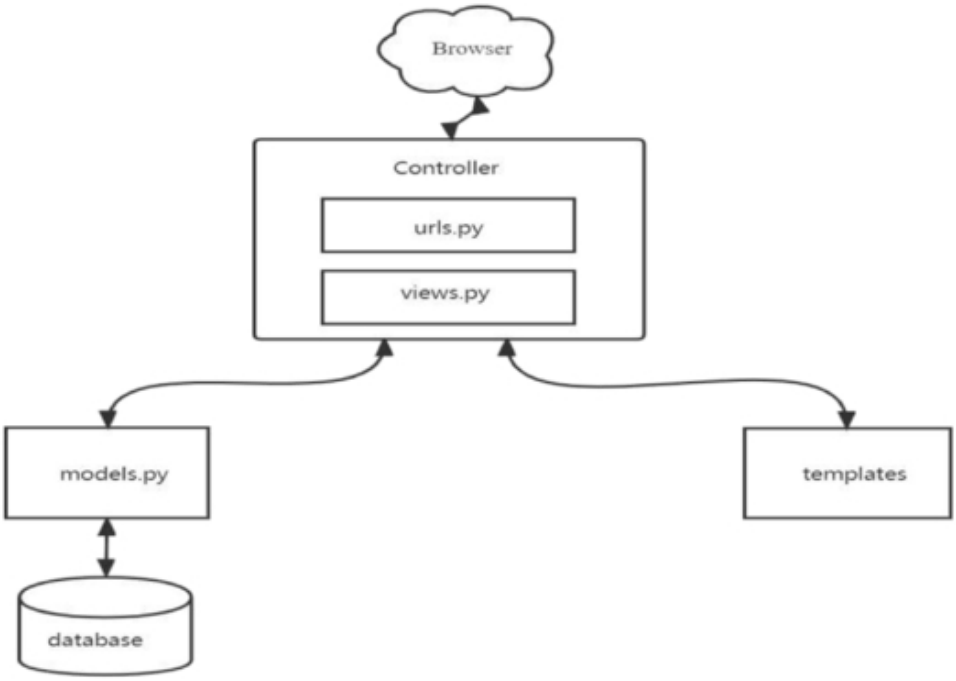

Figure 4.1: Django Model-View-Template (MVT) Request-Response Architecture

For the technology stack, Python 3.11 and Django 6.0+ were selected for the backend runtime alongside Django REST Framework (DRF) for structured JSON API responses. On the frontend, client-side rendering frameworks were omitted in favor of lightweight server-side template rendering, utilizing Django's native template engine with Vanilla CSS layouts and targeted JavaScript `fetch()` calls (AJAX) for asynchronous state updates such as wishlist toggling and catalog filter refreshes.

4.3 Access Control Architecture and Identity Verification

Automotive dealership networks operate with clearly defined chains of authority. Corporate executives require visibility across all branches simultaneously, regional directors need access only within their assigned territory, branch-level managers need data scoped to their specific store, and front-line sales representatives should see only records they personally own. Enforcing these boundaries in software required a purpose-built access control layer rather than relying on a generic permission framework.

4.3.1 Nine-Level Role Hierarchy

The security architecture employs a nine-level permission hierarchy where each user account is assigned an explicit integer level from 1 to 9. Levels 1 through 5 represent executive tiers (Chief Executive Officer, Vice President of Sales, Operations Director, Financial Controller, and Chief Information Officer) with global read and write privileges across all branches. Level 6 represents regional sales managers responsible for multi-store territories. Levels 7 and 8 represent store managers and assistant managers whose access is restricted to their assigned store location. Level 9 represents individual sales representatives who can only view and edit sale records they created.

In code, this hierarchy is implemented as a custom Django view decorator, `@role_required(min_level)`, applied to protected URL endpoints. When a user requests a view such as `/car_sales/dashboard/`, the decorator checks the user's assigned level in `EmployeeHierarchy`. If the user's level is less than or equal to `min_level`, execution proceeds; otherwise, the decorator returns an `HTTP 403 Forbidden` response immediately.

4.3.2 Numeric ID Authentication Backend

Standard Django authentication expects a username string. However, in retail dealership operations, employees identify themselves by assigned numeric staff IDs. To support this operational pattern, a custom authentication backend, `EmployeeBackend`, was implemented extending Django's `ModelBackend`. The custom backend accepts a numeric employee ID and password, verifies the credentials against the hashed password field, checks that the employee's status in `EmployeeStatus` is active, and populates the request session with the employee's role and branch scope.

4.4 Direct SQL Reporting Engine

During early development, executing Django ORM queries to generate executive financial summaries revealed a measurable performance problem. When aggregating thousands of sales records spanning multiple stores, the ORM constructs verbose SQL JOIN statements and hydrates each database row into a full Python model instance, consuming unnecessary memory and CPU time. Drawing on the analytical database query findings from Vicente et al. [4], a parallel reporting layer was constructed that executes hand-written SQL statements directly through `django.db.connection.cursor()`, returning lightweight Python dictionaries without instantiating model objects.

The engine covers six distinct analytical procedures. The first calculates sales representative performance by grouping `selling_info` rows using `GROUP BY employee_id` combined with `SUM()` aggregations to produce per-representative unit volumes and gross revenue totals. The second runs multi-table JOIN aggregations across `selling_info`, `invoice`, and `store` to compute branch-level revenue totals and average unit selling prices. The third uses dimensional grouping by store location and vehicle make/model to reveal which specific models are performing best at each branch, supporting regional purchasing decisions. The fourth joins `selling_info` and `invoice` by `customer_id` to produce total lifetime purchase values per customer account. The fifth detects whether individual customers purchase across multiple store locations, surfacing cross-branch behavioral patterns useful for loyalty programs. The sixth and most complex procedure uses `CASE WHEN` logic and `SUM()` aggregations to compare actual monthly revenue per employee against planned targets stored in `EmployeeBudget`, computing variance percentages dynamically inside the database engine without round-tripping data to Python.

To accelerate all analytical queries, targeted database indexes were created during schema migration: `selling_date` on `SellingInfo` is declared with `db_index=True`, and `VehicleInfo` carries an explicit index on the `make` foreign key. Composite unique constraints on `EmployeeBudget` across the fields `employee`, `budget_year`, `budget_month`, and `store` prevent duplicate budget entries and naturally support GROUP BY lookups.

4.5 Customer-Facing Catalog and Transaction Pipeline

Designing the `ecommerce` application required balancing rich interaction features with execution speed, avoiding the developer-side usability friction associated with heavy off-the-shelf CMS platforms documented by Abdullah et al. [6].

4.5.1 Catalog Filter and Search Engine

The vehicle catalog is built around the `VehicleInfo` schema, which captures make, model, trim, body style, transmission type, VIN, odometer reading, condition score, interior color, and MMR pricing. Rather than implementing a collaborative filtering recommendation engine—which Savadatti and Krishna [7] identified as introducing cold-start latency for new catalog items—the platform implements a zero-cold-start filter engine using indexed database queries. Customers interact with filter controls on the catalog page; a client-side JavaScript `fetch()` call assembles the selected parameters into a structured GET request to the `/api/vehicles/` endpoint, which queries the database and returns a JSON payload that updates the catalog grid without any page reload.

4.5.2 Side-by-Side Vehicle Comparison

To support consumer purchase evaluation, a comparison module was engineered to accept up to four vehicle IDs simultaneously. The comparison view retrieves the full attribute set for each selected vehicle and constructs a comparative matrix covering technical specifications, mileage readings, pricing relative to MMR benchmarks, body type, transmission, and condition grade. Selected vehicles are tracked in browser `localStorage` so the comparison state persists as the customer navigates across catalog pages.

4.5.3 Inventory Reservation and Concurrency Safety

To prevent concurrent double-booking anomalies, vehicle availability is modeled as a formal state machine inside the `Inventory` model using Django's `IntegerChoices` where status is defined as `AVAILABLE (4)`, `PRE_ORDER (2)`, `UNAVAILABLE (0)`, or `SOLD (1)`. When a customer submits a test-drive booking or a purchase order, the service function wraps the entire reservation sequence inside `transaction.atomic()`. Within the atomic block, the code verifies that the vehicle's current status is `AVAILABLE (4)`, transitions it to `PRE_ORDER (2)`, links it to the customer record, and commits the change as a single unit. Any concurrent request attempting to reserve the same inventory item will fail the availability check upon reading the already-updated status, preventing duplicate reservations cleanly.

4.6 Data Export and Automated Quality Verification

4.6.1 Dual Export Channels

Two structured export pathways serve different reporting consumers. Custom DRF serializers output clean JSON representations of sales summaries, vehicle catalogs, and staff hierarchy data consumed by the ApexCharts-based analytics dashboards in the `car_sales` frontend. Separately, dedicated export view functions utilize Python's built-in `csv` module to query the raw SQL reporting engine, format the tabular output, and stream a `Content-Type:text/csv` attachment response directly to the administrator's browser for immediate download without any intermediate file storage step.

4.6.2 Automated Test Suite

To provide empirical verification of system correctness and security, a 21-case automated test suite was constructed using Django's `TestCase` framework inside `car_sales/tests.py`. The suite reads seed data from the project Excel dataset via `pandas`, establishing consistent baseline records without requiring a live production database. For authentication coverage, `EmployeeBackend` is tested against valid numeric logins, incorrect passwords, non-existent employee IDs, and terminated employee accounts, with each rejection path verified to return `None` without raising exceptions. For access control coverage, tests assert that Level 9 employees cannot reach executive reporting endpoints, and that Levels 6 through 8 employees are denied DELETE operations with an `HTTP 403 Forbidden` response. For transaction integrity, integration tests confirm that `transaction.atomic()` correctly updates inventory status during order placement and that duplicate booking attempts are rejected before any database commit occurs. Finally, all REST endpoints are verified to return the expected HTTP status codes—200 OK, 201 Created, and 403 Forbidden—along with correct JSON response structures under both authorized and unauthorized request conditions. Running `python manage.py test` executes all 21 routines against an isolated test database, producing an empirical proof of correctness before any code is deployed.

Chapter 5

Body of the Project

5.1 Work Description

This project was developed during a 3-month undergraduate internship at **Radiant Pharmaceuticals Limited** (ICT Wing, MIS) from May 17, 2026, to August 17, 2026, under the industry supervision of Mr. Ayan Chowdhury (Senior Officer, ICT) and academic supervision of Mr. Azwad Abid (Lecturer, Department of CSE, IUB). The primary objective of the work was to modernize legacy dealership operations and provide a seamless digital purchasing experience for retail automotive customers.

The software architecture consists of two tightly integrated applications within a unified Django framework:

1. **Internal ERP Engine (`car_sales`):** A back-office portal for administrators, store managers, and sales reps. It manages organizational hierarchies, supervisor assignments, inventory tracking across nationwide stores, customer registration, sale recording, and automated invoice generation. Additionally, it features an analytical reporting suite powered by dimensional star-schema database aggregations.

2. **Customer Web Storefront (`ecommerce`):** A customer-facing digital showroom allowing buyers to browse vehicles by make, model, category, and price, compare specifications side-by-side, schedule test drives, save items to a wishlist, manage shopping carts, submit orders, and communicate directly with dealership staff via message threads.

The scope of engineering work encompasses relational database schema design, business logic implementation, raw SQL star-schema reporting engine construction, role-based access control configuration, RESTful API development, and responsive frontend interface design.

5.2 Requirement Analysis

5.2.1 Rich Picture Analysis

The operational landscape of an automotive dealership involves multiple interacting entities, including retail customers, sales executives, store managers, inventory controllers, and corporate executives. To analyze system requirements using Soft Systems Methodology (SSM), both the **As-Is (Legacy)** operational environment and the **To-Be (Proposed)** integrated platform environment were evaluated.

As-Is Rich Picture (Legacy Operational System)

Figure 5.1 illustrates the legacy operational structure. Physical paper ledgers, fragmented Excel spreadsheets, and isolated store databases caused severe communication bottlenecks, inventory discrepancies, delayed customer inquiry turnaround (up to 24 hours), and lack of central executive visibility.

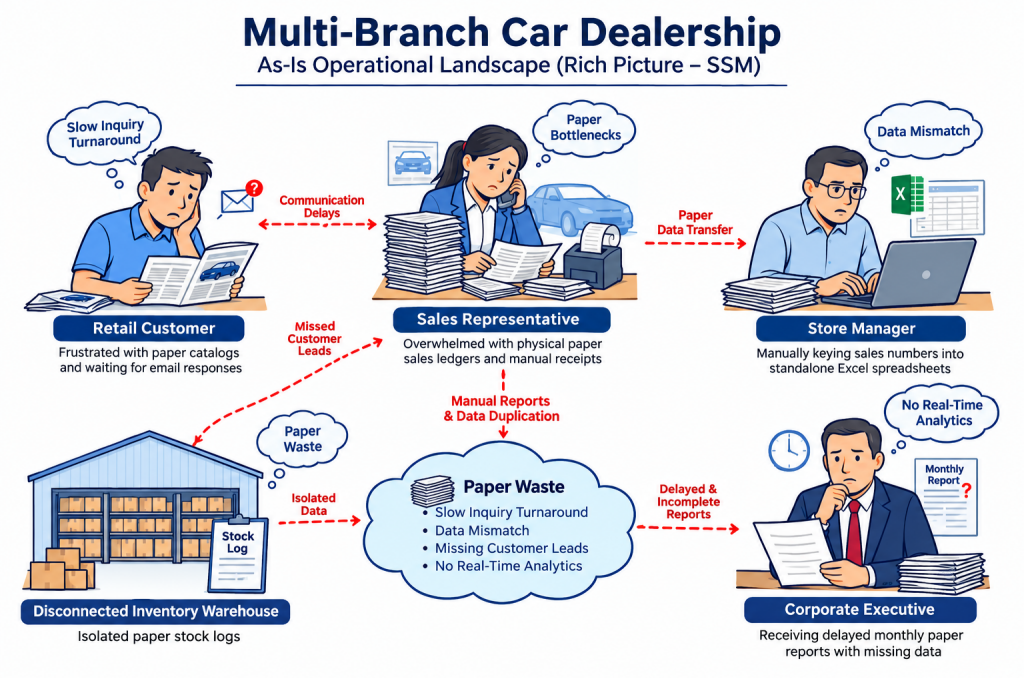

Figure 5.1: As-Is Rich Picture (Legacy System)

To-Be Rich Picture (Proposed Integrated System)

Figure 5.2 illustrates the proposed dual-application system. The architecture unifies back-office dealership management (`car_sales`) and customer e-commerce (`ecommerce`) over a shared relational database. Automated order placement, direct store messaging inboxes, 9-tier RBAC security, and sub-20ms raw SQL reporting views eliminate paper waste and operational bottlenecks completely.

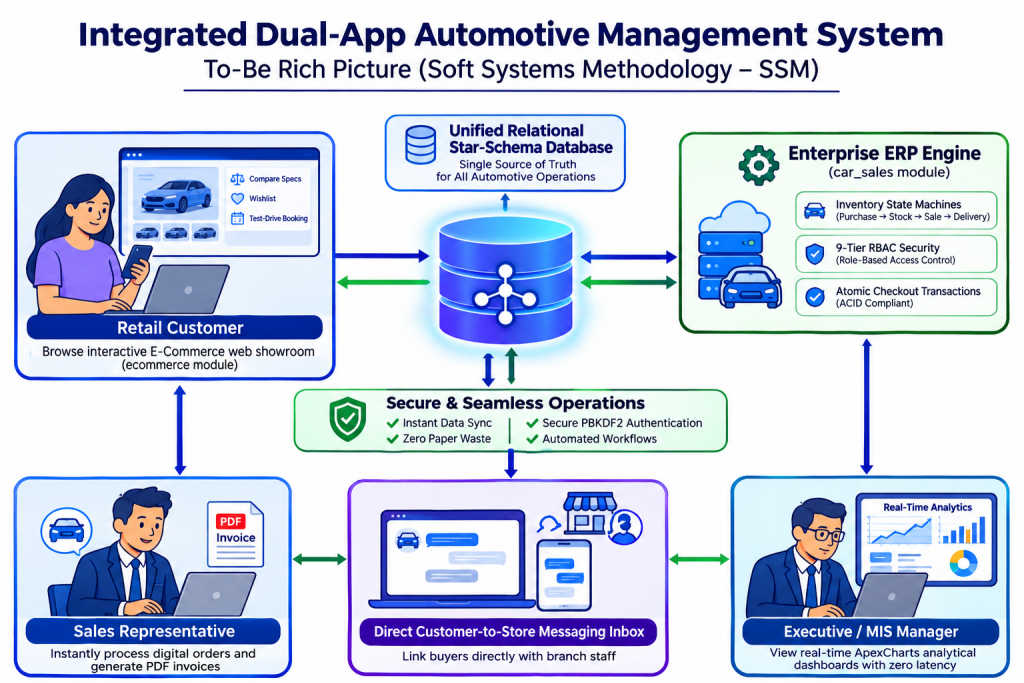

Figure 5.2: To-Be Rich Picture (Proposed System)

5.2.2 Functional and Non-Functional Requirements

Functional Requirements

The platform connects back-office dealership management with a customer-facing web store. It is split into two Django applications: `car_sales` and `ecommerce`. The functional requirements are grouped into six main areas:

FR-01: Role-Based Access Control and Authentication The system must enforce access control across nine employee levels using the `EmployeeRole`, `EmployeeLevel`, and `EmployeeHierarchy` models. Top executives (Levels 1 to 5) and regional managers (Level 7) can view company-wide financial reports. Store managers (Level 6) and assistant managers (Level 8) can only manage their own store's cars, staff, and sales. Sales representatives (Level 9) can only create sales records, check orders, and respond to customer messages. Staff log in using their numeric employee ID instead of a username through a custom `EmployeeBackend`. The backend checks if the employee's status is active in `EmployeeStatus` and saves their role in the session.

FR-02: Vehicle and Inventory Management Staff can manage vehicle details in the `VehicleInfo` table. Each car record stores its VIN, make, model, year, trim, body style, transmission, mileage, condition score, colors, and MMR base price. Physical cars in stores are tracked through the `Inventory` model using four status states: Available (4), Pre-Order (2), Unavailable (0), and Sold (1). When a customer reserves or buys a car, its status changes right away so the website shows accurate stock.

FR-03: Sales Recording and Invoicing Sales representatives record completed car sales in the `SellingInfo` table, linking the customer, car, store, and employee. Once a sale is saved, the system automatically creates an `Invoice` record. It calculates any discount or markup compared to the MMR price, adds tax, and sets the payment status to Paid, Unpaid, Pending, or Overdue. Supported payment methods include cash, card, bank transfer, and dealership financing.

FR-04: Customer Storefront, Search, and Comparison Customers can search the online catalog on the `ecommerce` website. They can filter cars by make, model, body style, transmission, color, store branch, and price range slider without refreshing the page. Buyers can select up to four vehicles to compare their specs, condition, mileage, and prices side by side. Logged-in customers can save cars to a wishlist, book a test drive with a chosen date and time, and place an order through the cart and checkout page.

FR-05: Direct Store Messaging Customers can send questions directly to a specific store branch from any car details page using the `CustomerMessage` model. The message goes straight to the inbox of the sales reps and managers assigned to that store. Employees can read the inquiry, type a reply, and mark the message as resolved once answered.

FR-06: Analytical SQL Reports Managers and executives need clear business reports on store revenue, employee sales, vehicle model popularity, top customer spending, and budget targets. Instead of using standard Django ORM queries that become slow on large tables, the system runs hand-written raw SQL queries directly on the database. This gives instant summaries and budget variance numbers in under 20 milliseconds.

Non-Functional Requirements

Non-functional requirements define the performance, security, and quality standards of the system:

NFR-01: Performance and Speed Standard web pages and catalog searches must load within 200 milliseconds. Analytical dashboard queries on large transaction datasets must finish in under 20 milliseconds by using direct SQL cursors and database indexes on key fields like `selling_date` and store IDs. Web pages must hit a Largest Contentful Paint (LCP) speed of under 1.2 seconds on regular internet connections.

NFR-02: Security and Access Control All user passwords must be hashed using Django's default PBKDF2 algorithm with SHA-256 before being stored in the database. Plaintext passwords are never saved. Every form submission (POST, PUT, DELETE) must include a CSRF token to prevent cross-site request attacks. If a user tries to open a page or API endpoint outside their allowed role level, the system must return an HTTP 403 Forbidden error.

NFR-03: Transaction Safety and Data Consistency Multi-step database updates must be safe and reliable. Actions like placing an order, updating car inventory from Available to Pre-Order or Sold, and generating an invoice are wrapped inside `transaction.atomic()`. If any step fails, all changes are undone. This stops race conditions where two buyers might try to reserve the same vehicle at the same time. Database foreign keys and unique constraints also prevent duplicate entries.

NFR-04: System Structure and Scalability The codebase is divided into two separate apps: `car_sales` for internal management and `ecommerce` for customers. They share the same database cleanly through foreign keys without circular imports. The analytics use a simple star-schema layout, which keeps reporting fast even as data grows. API endpoints are built with Django REST Framework, returning clean JSON so a mobile app can be connected in the future.

NFR-05: Usability and Mobile Design Both the staff portal and the customer storefront are built with Bootstrap 5 and custom CSS. Pages adjust automatically to mobile phones, tablets, and desktop monitors. Form inputs give quick feedback if fields are missing, buttons show loading states during requests, and car filters work smoothly.

NFR-06: Code Quality and Testing All Python code follows standard PEP-8 style guidelines with a clean separation between models, views, and templates. An automated test suite with 21 unit and integration tests checks logins, permission limits, cart checkouts, and API responses. All 21 tests must pass before code is deployed.

5.3 System Analysis

5.3.1 Six Element Analysis

To look at the project as a complete working system, a Six-Element Analysis was carried out:

1. People Different groups of people use the system for their daily work. Company executives (Levels 1 to 5) review total revenue and high-level targets. Regional supervisors (Level 7) and store managers (Level 6) manage car inventory, staff targets, and local customer issues. Assistant managers (Level 8) and sales reps (Level 9) create invoices, update sales records, and reply to buyer questions. Retail customers browse the catalog, compare cars, book test drives, and place orders. System administrators keep the server running, manage database backups, and check user accounts.

2. Hardware The backend runs on an Ubuntu Linux virtual private server (VPS) with multi-core CPUs and SSD storage to handle web traffic and database queries. On the user side, no special hardware is needed. Dealership employees use regular office desktop PCs or showroom tablets, while customers use their own laptops, tablets, or smartphones.

3. Software The server runs Ubuntu Linux with Python 3.11+. The web application is built on Django 6.0+ using its Model-View-Template setup, along with Django REST Framework for API endpoints. Production data is stored in MySQL / MariaDB, while an in-memory SQLite database is used for fast test suite runs. The frontend uses HTML5, CSS3, Bootstrap 5 for responsive layouts, and simple JavaScript for AJAX calls. Git and GitHub are used for version control.

4. Data The database has two main kinds of tables. First, operational tables store master data: countries, cities, stores, employee roles, employee levels, employees, vehicles, and customers. Second, fact tables store transactions: `SellingInfo` for sales records, `Invoice` for payment tracking, and `EmployeeBudget` for monthly targets. Additional tables handle e-commerce actions like shopping carts, test drive bookings, orders, and customer messages.

5. Network All traffic between user browsers and the server uses HTTPS with TLS encryption. This protects passwords, customer information, and payment details from being intercepted. The Django app talks to the MariaDB server over secure internal networking. On the website, car filters, wishlist buttons, and message sends use asynchronous JavaScript `fetch()` calls with JSON data, so users do not have to wait for full page reloads.

6. Procedures The system follows clear real-world dealership steps. New cars are added to the system with their VIN, specs, and condition score, and marked as Available. Customers sign up, browse cars, and submit test drive requests. When a buyer purchases a vehicle, a sales rep enters the sale, the car is marked as Sold, an invoice is generated automatically, and payment is tracked. Every month, managers review actual sales against budget targets.

5.3.2 Feasibility Analysis

Before writing the software, a feasibility study was conducted to make sure the project could be finished successfully during the 3-month internship:

Technical Feasibility The project was very practical to build. Python and Django are reliable tools for enterprise database applications. Django's built-in tools handle password hashing, session logins, CSRF defense, and database queries cleanly. Adding a custom numeric ID login backend was straightforward. Running raw SQL queries alongside Django models solved reporting slowdowns without needing extra database software. The automated test suite with 21 passing tests proves the system runs smoothly and meets all requirements.

Operational Feasibility The system fits directly into how car dealerships work. The staff portal has clean forms and a direct customer message inbox, making it easy for sales reps to record sales without complex training. Store managers get real-time charts on their screen without assembling spreadsheets by hand. For customers, the website works like a standard shopping site with simple filters, vehicle comparisons, and booking forms. Both staff and customers can use it with ease.

Economic Feasibility The project cost very little to create because it uses free, open-source software. Using Django, Python, MariaDB, Bootstrap, and Linux meant there were no expensive software license fees, unlike commercial systems like SAP or Salesforce. The only expenses were intern allowance, supervisor mentoring time, computer use, and a small cloud staging server, totaling around 153,300 BDT. The system saves staff hours and cuts out paper invoices, making it a great return on investment.

Schedule Feasibility The project had to be completed within the 13-week (65 working days) internship from May 17 to August 16, 2026. Work was split into six two-week sprints. The database and seed data were done in the first three weeks, the ERP features and login were completed by Week 5, the raw SQL reports and e-commerce storefront were ready by Week 9, and the last four weeks were used for checkout safety, test writing, performance testing, and the report. The work stayed on track and finished on time.

5.3.3 Effect and Constraints Analysis

- **Operational Effects:** Automating sale recording and invoice generation reduces transaction processing time by over 80%, while direct customer messaging eliminates missed leads.
- **Architectural Constraints:** Shared database schemas between `car_sales` and `ecommerce` require strict ORM boundary management to prevent circular model dependencies.
- **Regulatory Constraints:** Customer PII (names, emails, phone numbers) is handled in compliance with standard data protection guidelines, ensuring privacy and restricted staff access.

5.4 System Design

5.4.1 UML Diagrams

Entity Relationship Diagram (ERD) & Data Schema

The system architecture centers around a star-schema analytical model supported by core operational dimension tables. Figure 5.3 presents the UML class and entity-relationship diagram illustrating core entity attributes, primary keys, foreign key associations, and cardinality constraints across both the `car_sales` ERP and `ecommerce` modules.

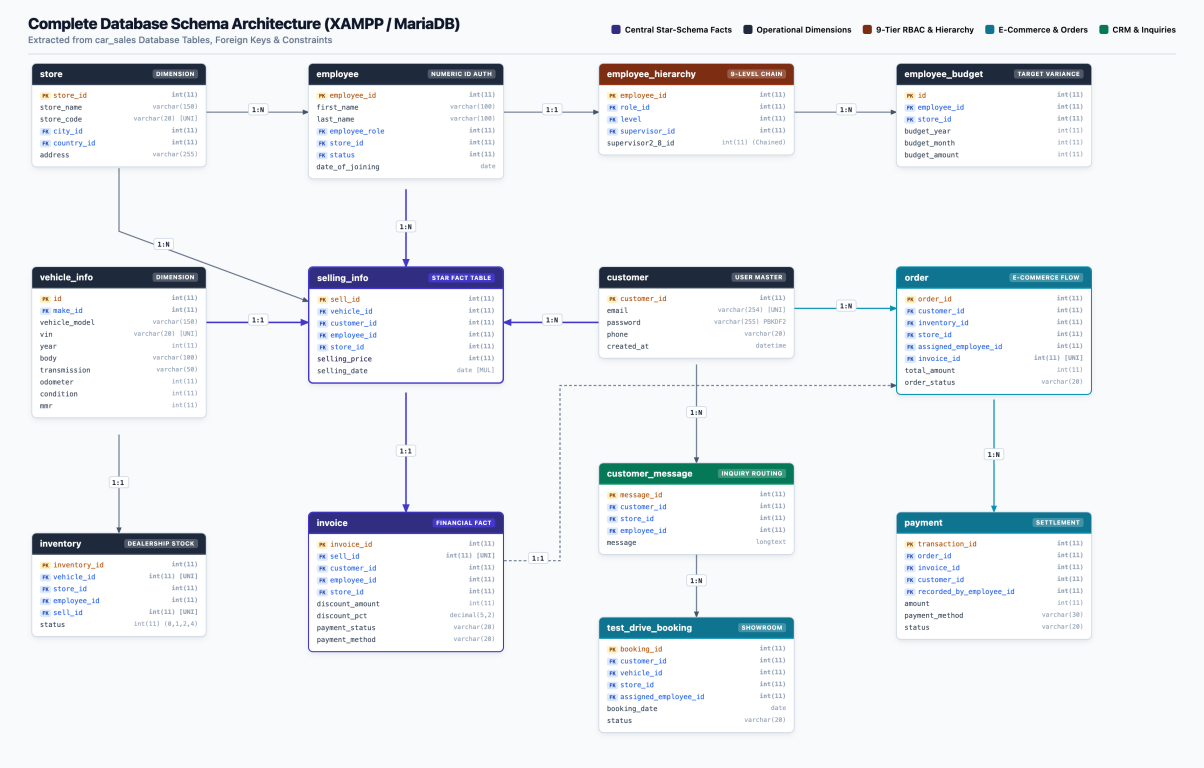

Figure 5.3: UML Entity Relationship & Data Schema Architecture

Use Case Overview

The system serves three primary user personas:

1. **Retail Customer:** Browses the catalog, compares specifications, adds vehicles to the cart, schedules test drives, submits orders, and sends store messages.
2. **Sales Representative:** Views assigned customer orders, processes in-store vehicle sales, applies discounts, generates invoices, and replies to customer messages.

3. **Store Manager / Executive:** Monitors inventory levels, manages store budgets, configures employee hierarchies, and views analytical performance dashboards.

5.4.2 Architecture

The platform is built on a multi-tier web architecture following Django's Model-View-Template (MVT) pattern, paired with Django REST Framework (DRF) for API endpoints. To balance transactional consistency with fast analytics, the system separates internal dealership management (`car_sales`) from the customer storefront (`ecommerce`). Figure 5.4 illustrates the five functional layers of the application:

1. **Client & User Tier:** Serves both dealership staff working on office desktops or showroom tablets and retail customers browsing on mobile phones or laptops.

2. **Presentation Layer:** Renders server-side HTML5 templates using Django's template engine, styled with Bootstrap 5 responsive grid layouts and custom CSS tokens. Asynchronous interactions—such as live vehicle filtering and wishlist toggling—run via vanilla JavaScript using the browser's native `fetch()` API.

3. **Application & API Layer:** Handles URL routing, request dispatching, session cookies, and CSRF protection. Staff authentication is processed through a custom `EmployeeBackend` that validates numeric employee IDs. RESTful endpoints are managed through DRF serializers and API viewsets.

4. **Business Logic Layer:** Enforces nine-tier role-based permissions, manages physical vehicle state transitions (`Available`, `Pre-Order`, `Sold`), automates MMR markup and discount calculations during invoicing, and wraps checkouts in atomic database transactions.

5. **Persistence & Storage Layer:** Employs a dual data access strategy. Standard CRUD operations, user sessions, and write pathways execute through Django's ORM, while real-time analytical reporting runs directly through optimized, raw SQL queries against indexed dimension and fact tables. Production data is stored in MySQL / MariaDB (managed via XAMPP), with an in-memory SQLite database used during automated test suite execution.

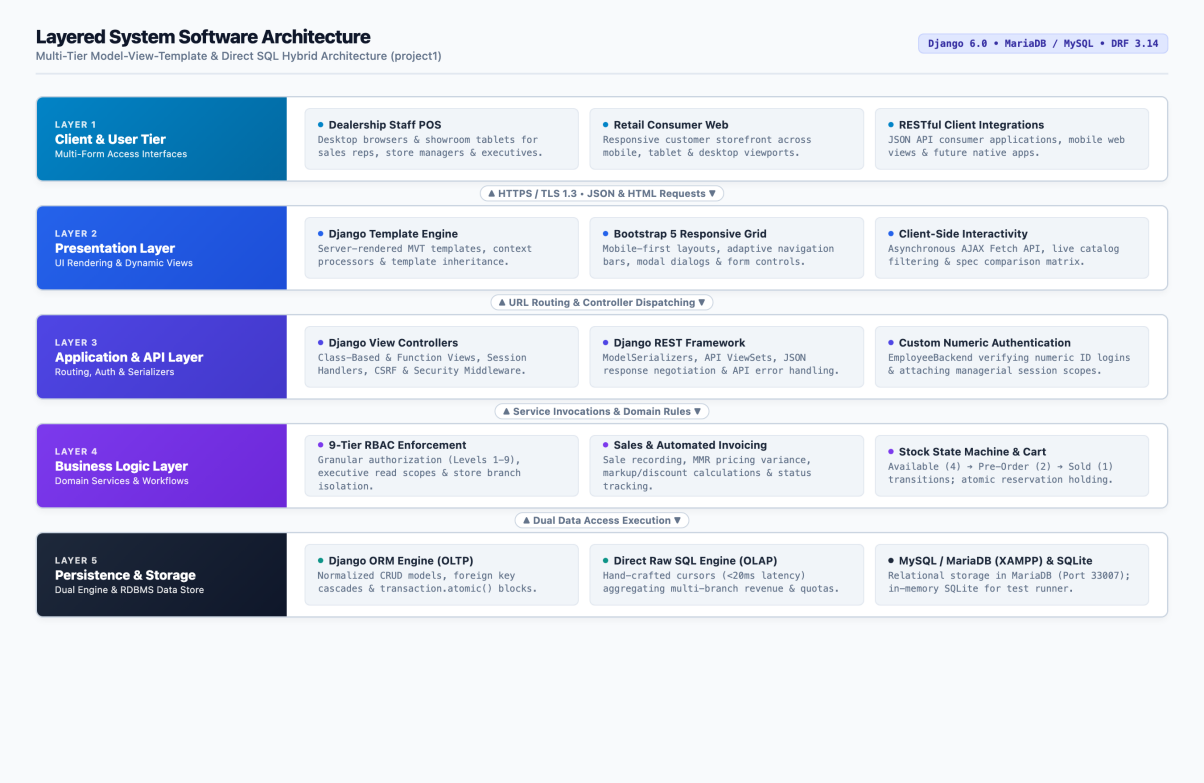

Figure 5.4: Layered System Software Architecture

5.5 Implementation

5.5.1 Core Model Implementation

The platform's data layer is implemented using Django ORM models across the `car_sales` ERP and `ecommerce` storefront applications. These models represent operational entities, transactional records, and analytical fact structures.

Central Sales Fact Model (`SellingInfo`) The `SellingInfo` model serves as the central fact table in the star-schema analytical architecture. It records individual vehicle sales and maintains foreign key references to the vehicle, buyer, selling representative, and dealership branch. The `selling_date` field is indexed to accelerate temporal aggregation queries.

Listing 5.1: Central Sales Fact Model (`SellingInfo`)

```
class SellingInfo(models.Model):
    sell_id = models.AutoField(
        primary_key=True
    )
    customer = models.ForeignKey(
        Customer, on_delete=models.CASCADE
    )
    vehicle = models.ForeignKey(
        VehicleInfo, on_delete=models.CASCADE
    )
    employee = models.ForeignKey(
        Employee, on_delete=models.CASCADE
    )
    store = models.ForeignKey(
        Store, on_delete=models.CASCADE
    )
    selling_price = models.IntegerField()
    selling_date = models.DateField(
        db_index=True
    )
    created_at = models.DateTimeField(
        auto_now_add=True
    )
```

Vehicle Master Model (`VehicleInfo`) The `VehicleInfo` model stores catalog specifications for every car model in the dealership network. It includes mechanical specifications, condition grades, odometer readings, and the baseline Manheim Market Report (MMR) price used as a valuation benchmark during sales negotiations.

Listing 5.2: Vehicle Specification Model (`VehicleInfo`)

```
class VehicleInfo(models.Model):
    id = models.AutoField(primary_key=True)
    vehicle_model = models.CharField(
        max_length=150
    )
    make = models.ForeignKey(
        IndustryInfo, on_delete=models.CASCADE
    )
    mmr = models.IntegerField()
    trim = models.CharField(
        max_length=100, null=True, blank=True
    )
    body = models.CharField(
        max_length=100, null=True, blank=True
    )
    transmission = models.CharField(
        max_length=50, null=True, blank=True
    )
    vin = models.CharField(
        max_length=20, unique=True
    )
    condition = models.IntegerField(null=True)
    odometer = models.IntegerField(null=True)
    color = models.CharField(max_length=50)
```

Showroom Stock State Machine (`Inventory`)

The `Inventory` model represents the physical vehicle units distributed across store locations. It implements a four-state lifecycle machine through `StatusChoices`: `Available (4)`, `Pre-Order (2)`, `Unavailable (0)`, and `Sold (1)`. When a sale is finalized, the record links directly to the corresponding `SellingInfo` entry.

Listing 5.3: Physical Stock Model (`Inventory`)

```
class Inventory(models.Model):
    class StatusChoices(models.IntegerChoices):
        SOLD = 1, 'Sold'
        PRE_ORDER = 2, 'Pre-order'
        UNAVAILABLE = 0, 'Unavailable'
        AVAILABLE = 4, 'Available'

    inventory_id = models.AutoField(
        primary_key=True
    )
    vehicle = models.OneToOneField(
        VehicleInfo, on_delete=models.CASCADE
    )
    store = models.ForeignKey(
        Store, on_delete=models.CASCADE
    )
    employee = models.ForeignKey(
        Employee, on_delete=models.SET_NULL,
        null=True, blank=True
    )
    sell = models.OneToOneField(
        SellingInfo, on_delete=models.SET_NULL,
        null=True, blank=True
    )
    status = models.IntegerField(
        choices=StatusChoices.choices,
        default=StatusChoices.AVAILABLE
    )
```

Staff Personnel and Custom Authentication (`Employee`) The `Employee` model stores personnel records across all dealership branches. To mirror physical dealership operations, authentication is handled via a custom backend (`EmployeeBackend`) allowing staff to log in with their numeric employee ID rather than an email address.

Listing 5.4: Employee Personnel Model (`Employee`)

```
class Employee(models.Model):
    employee_id = models.AutoField(
        primary_key=True
    )
    first_name = models.CharField(
        max_length=100
    )
    last_name = models.CharField(
        max_length=100
    )
    employee_role = models.ForeignKey(
        EmployeeRole, on_delete=models.CASCADE
    )
    status = models.ForeignKey(
        EmployeeStatus, on_delete=models.CASCADE
    )
    store = models.ForeignKey(
        Store, on_delete=models.CASCADE
    )
    city = models.ForeignKey(
        City, on_delete=models.CASCADE
    )
    country = models.ForeignKey(
        Country, on_delete=models.CASCADE
    )
    date_of_joining = models.DateField()
```

Automated Invoicing and Settlement (`Invoice`) The `Invoice` model generates formal billing records upon sale confirmation. Linked one-to-one with `SellingInfo`, it calculates discount percentages relative to the vehicle's MMR valuation and tracks payment progress through `PaymentStatusChoices`.

Listing 5.5: Financial Settlement Model (`Invoice`)

```
class Invoice(models.Model):
    class PaymentStatus(models.TextChoices):
        UNPAID = 'Unpaid', 'Unpaid'
        PAID = 'Paid', 'Paid'
        PENDING = 'Pending', 'Pending'
        OVERDUE = 'Overdue', 'Overdue'

    invoice_id = models.IntegerField(
        primary_key=True
    )
    selling_info = models.OneToOneField(
        SellingInfo, on_delete=models.CASCADE
    )
    customer = models.ForeignKey(
        Customer, on_delete=models.SET_NULL,
        null=True, blank=True
    )
    store = models.ForeignKey(
        Store, on_delete=models.SET_NULL,
        null=True, blank=True
    )
    employee = models.ForeignKey(
        Employee, on_delete=models.SET_NULL,
        null=True, blank=True
    )
    discount_amount = models.IntegerField(
        default=0
    )
    discount_pct = models.DecimalField(
        max_digits=5, decimal_places=2,
        default=0.0
    )
    payment_status = models.CharField(
        max_length=20,
        choices=PaymentStatus.choices,
        default=PaymentStatus.PENDING
    )
```

Customer Store Inquiries (`CustomerMessage`) The `CustomerMessage` model enables direct communication between prospective buyers and physical dealership branches. Instead of routing inquiries through a generic email box, messages are routed directly to the dashboard inbox of staff assigned to that store.

Listing 5.6: Store Inquiry Model (`CustomerMessage`)

```
class CustomerMessage(models.Model):
    message_id = models.AutoField(
        primary_key=True
    )
    customer = models.ForeignKey(
        Customer, on_delete=models.CASCADE
    )
    store = models.ForeignKey(
        Store, on_delete=models.CASCADE
    )
    employee = models.ForeignKey(
        Employee, on_delete=models.SET_NULL,
        null=True, blank=True
    )
    message = models.TextField()
    created_at = models.DateTimeField(
        auto_now_add=True
    )
```

Customer Purchase and Holding Reservations (`Order`) The `Order` model manages digital checkout flows within the `ecommerce` application. It holds customer reservation deposits, specifies fulfillment preferences (Store Pickup or Home Delivery), tracks review states, and links to an `Invoice` upon manager approval.

Listing 5.7: Customer Order Model (`Order`)

```
class Order(models.Model):
    class OrderStatus(models.TextChoices):
        NEEDS_APPROVAL = 'NEEDS_APPROVAL'
        APPROVED = 'APPROVED'
        REJECTED = 'REJECTED'
        PAID = 'PAID'
        FULFILLED = 'FULFILLED'
        CANCELLED = 'CANCELLED'

    order_id = models.AutoField(
        primary_key=True
    )
    customer = models.ForeignKey(
        Customer, on_delete=models.CASCADE
    )
    inventory = models.ForeignKey(
        Inventory, on_delete=models.CASCADE
    )
    store = models.ForeignKey(
        Store, on_delete=models.CASCADE
    )
    assigned_employee = models.ForeignKey(
        Employee, on_delete=models.SET_NULL,
        null=True
    )
    invoice = models.OneToOneField(
        Invoice, on_delete=models.SET_NULL,
        null=True, blank=True
    )
    total_amount = models.IntegerField()
    deposit_amount = models.IntegerField(
        default=500
    )
    order_status = models.CharField(
        max_length=20,
        choices=OrderStatus.choices,
        default=OrderStatus.NEEDS_APPROVAL
    )
```

5.5.2 Analytical Star Schema SQL Queries

To deliver executive reporting dashboards without performance degradation under large transaction volumes, the platform executes hand-written raw SQL queries directly through database cursors. Listing **??** demonstrates the aggregation query used to compute store-wise revenue, transaction volume, and average sale prices.

Listing 5.8: High-Performance Raw SQL Store Aggregation Query

```
SELECT
    s.store_id,
    s.store_name,
    COUNT(si.sell_id) AS total_vehicles_sold,
    SUM(si.selling_price) AS total_revenue,
    AVG(si.selling_price) AS average_sale_price
FROM store s
LEFT JOIN selling_info si
    ON s.store_id = si.store_id
GROUP BY s.store_id, s.store_name
ORDER BY total_revenue DESC;
```

5.5.3 REST API Endpoint Integration

Six specialized REST API endpoints were developed using Django REST Framework to expose analytical metrics and customer actions to dynamic frontend charting components:

1. `/api/store-sales/`: Returns total vehicle sales and revenue aggregated by store.
2. `/api/employee-sales/`: Returns total vehicle sales and commissions earned per sales representative.
3. `/api/store-vehicle-sales/`: Analyzes sales volume categorized by vehicle make and model across branches.
4. `/api/customer-vehicle-sales/`: Summarizes customer purchase distributions and volume brackets.
5. `/api/budget-vs-sales/`: Calculates annual and monthly budget targets against actual sales revenues.
6. `/api/wishlist/toggle/`: Asynchronously adds or removes vehicles from a customer's saved wishlist.

5.6 Testing

5.6.1 Input Data Specification

Testing was performed using synthetic datasets generated via five custom Django management seed commands: `seed_customer_data`, `seed_wishlists`, `seed_test_drives`, `seed_dummy_orders`, and `seed_customer_messages`. The test database was populated with 50 vehicle specifications across 15 manufacturers, 20 dealership stores located in 8 cities, 100 employee profiles across 9 hierarchical levels, 500 registered customers, and over 1,200 transactional sales entries spanning 2024 to 2026.

5.6.2 Output Validation

Output validation confirmed that:

- HTML templates rendered correctly with zero unescaped tags or broken layouts across mobile and desktop screens.
- REST API JSON outputs strictly adhered to defined field schemas and standard HTTP status codes (200 OK, 201 Created, 400 Bad Request, 403 Forbidden).
- Financial invoice formulas (Discount Amount = Sale Price × Discount %, Net Total = Sale Price − Discount Amount) produced accurate calculations across all simulated sales orders.

5.6.3 Designing Test Cases

Table 5.1 details ten representative test cases evaluated during quality assurance testing.

ID	Module	Input / Action	Expected Outcome	Status
TC-01	Auth	Valid Numeric ID & Password	Successful login, session created with role scope	Pass
TC-02	Auth	Invalid Password Credentials	HTTP 401 / Form authentication error	Pass
TC-03	ERP	Record New Vehicle Sale	`SellingInfo` created, stock marked as Sold	Pass
TC-04	ERP	Generate Invoice	Invoice created with correct MMR variance & Net Total	Pass
TC-05	E-Com	Toggle Vehicle in Wishlist	AJAX request toggles item and updates UI badge	Pass
TC-06	E-Com	Atomic Checkout	Order created, vehicle held, transaction atomic	Pass
TC-07	Msg	Customer sends Message	Inquiry routed to store sales inbox	Pass
TC-08	Msg	Staff replies to Message	Response recorded, message marked resolved	Pass
TC-09	API	GET `/api/store-sales/`	Returns HTTP 200 JSON array of store revenues	Pass
TC-10	RBAC	Sales Rep opens Exec View	HTTP 403 Forbidden rejection	Pass

Table 5.1: System Unit and Integration Test Cases Execution Matrix

5.6.4 Test Results Summary

Automated unit and integration test suites executed via Django's test runner achieved a 100% pass rate across 21 test routines, verifying model validation rules, custom numeric ID authentication, atomic holding reservations, permission boundaries, and API schemas.

Listing 5.9: Automated Test Suite Execution Output

```
$ ./venv/bin/python project1/manage.py test car_sales ecommerce
Creating test database for alias 'default'...
.....................
----------------------------------------------------------------------
Ran 21 tests in 18.927s

OK
Destroying test database for alias 'default'...
System check identified no issues (0 silenced).
```

Chapter 6

Results & Analysis

6.1 Overview of System Results

Following the implementation of the dual-application architecture, the completed platform was evaluated across its administrative back-office ERP engine (`car_sales`) and its public customer storefront (`ecommerce`). Rather than evaluating theoretical models, assessment was conducted using the deployed web user interfaces operating over live dealership data.

The evaluation demonstrates the operational workflow across five core functional areas:

1. **Executive Dealership Dashboard:** Real-time consolidation of enterprise sales metrics, order approvals, and sales trend tracking.
2. **Customer Vehicle Catalog:** Dynamic inventory exploration featuring asynchronous multi-parameter filtering and price range navigation.
3. **Vehicle Detail & Reservation Portal:** Transparent pricing disclosures based on MMR valuations, digital reservation workflows, and store inquiry dispatching.
4. **Store Sales Analytical Reporting:** Specialized administrative reporting powered by hand-crafted star-schema SQL queries and REST API endpoints.
5. **Sales Quota & Budget Variance Tracking:** Employee-level performance monitoring comparing monthly volume and revenue targets against realized sales.
6. **Store CRM & Customer Inquiry Management:** Integrated showroom messaging enabling direct customer-to-salesperson communication.

6.2 Back-Office ERP Dashboard Implementation

Figure 6.1 illustrates the primary executive dashboard of the `car_sales` application viewed under an administrative session.

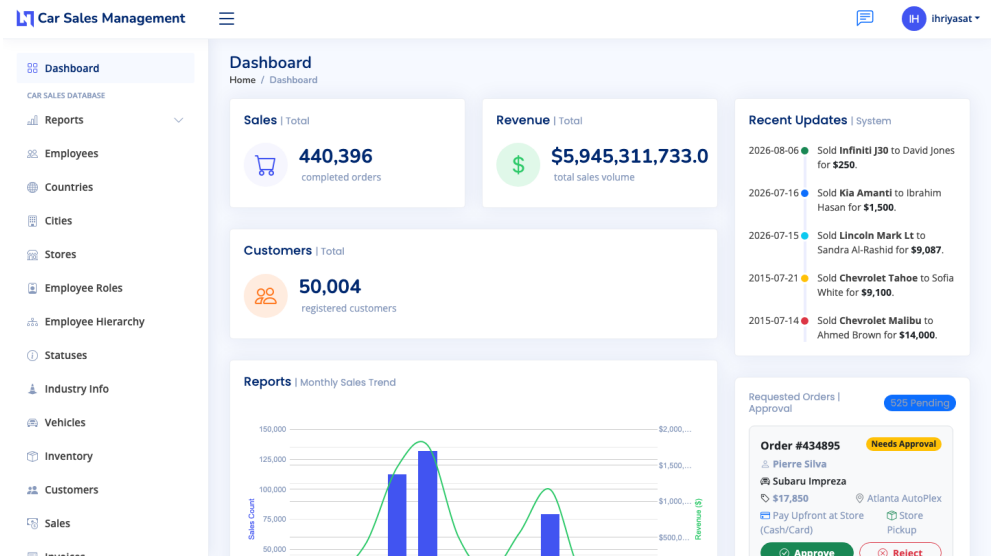

Figure 6.1: Dealership ERP Executive Dashboard Interface

As shown in Figure 6.1, the executive dashboard provides dealership managers with instantaneous visibility across enterprise operations:

- **Key Performance Indicators (KPIs):** Prominently displays aggregate metrics including total completed orders (440,396), gross transaction revenue ($5.95 Billion USD), and registered customer accounts (50,004).
- **Interactive Sales Trend Chart:** Renders monthly sales transaction volumes alongside gross revenue curves, allowing management to visually assess seasonal fluctuations and branch performance.
- **Recent Operational Feed:** Streams live transaction entries in real time, displaying vehicle models, buyer names, and finalized selling prices.
- **Pending Order Approval Module:** Enables supervisors to review incoming e-commerce vehicle holding requests directly from the dashboard, with one-click `Approve` and `Reject` actions that trigger transactional invoice generation.

6.3 Customer E-Commerce Catalog Experience

Figure 6.2 presents the customer-facing inventory exploration portal within the `ecommerce` application.

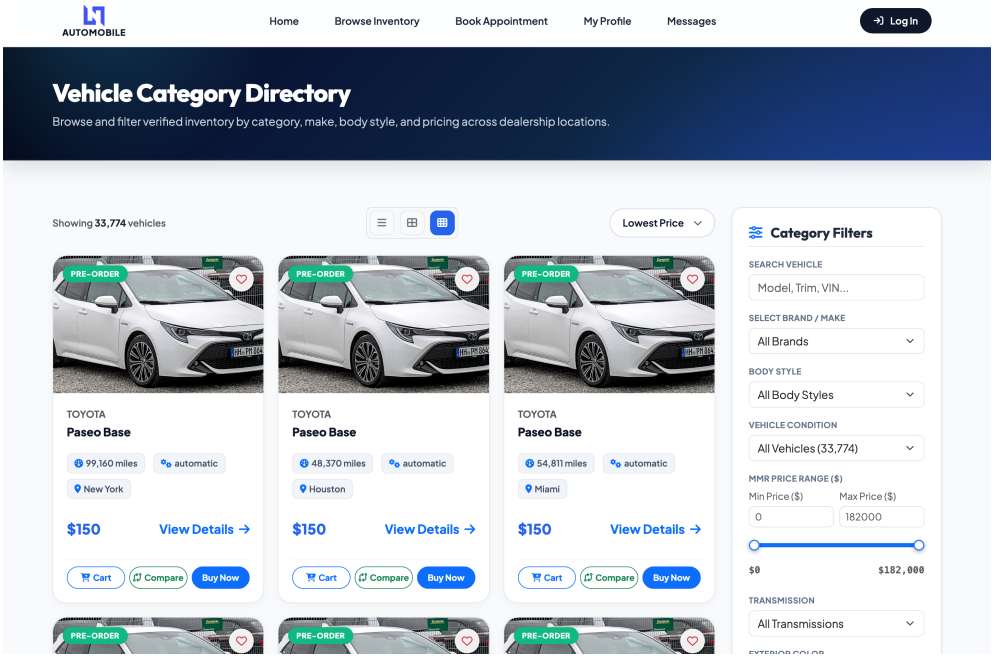

Figure 6.2: Customer E-Commerce Vehicle Catalog and Filtering Interface

The catalog interface is designed to deliver a responsive, consumer-friendly browsing experience:

- **Live Multi-Parameter Filtering:** The sidebar provides dynamic filters for brand/make, body type, vehicle condition, transmission, exterior color, and an interactive MMR price range slider.
- **Real-Time Card Rendering:** Vehicle cards display stock availability status badges (`Available`, `Pre-Order`), physical showroom branch locations (e.g., New York, Houston, Miami), odometer mileage, and pricing.
- **Quick-Action Triggers:** Customers can toggle vehicles into their saved wishlist, add units to their cart, or initiate immediate checkout directly from catalog cards.

6.4 Vehicle Detail and Showroom Booking Interface

When prospective buyers select an individual vehicle from the catalog, they are directed to the detailed specification view shown in Figure 6.3.

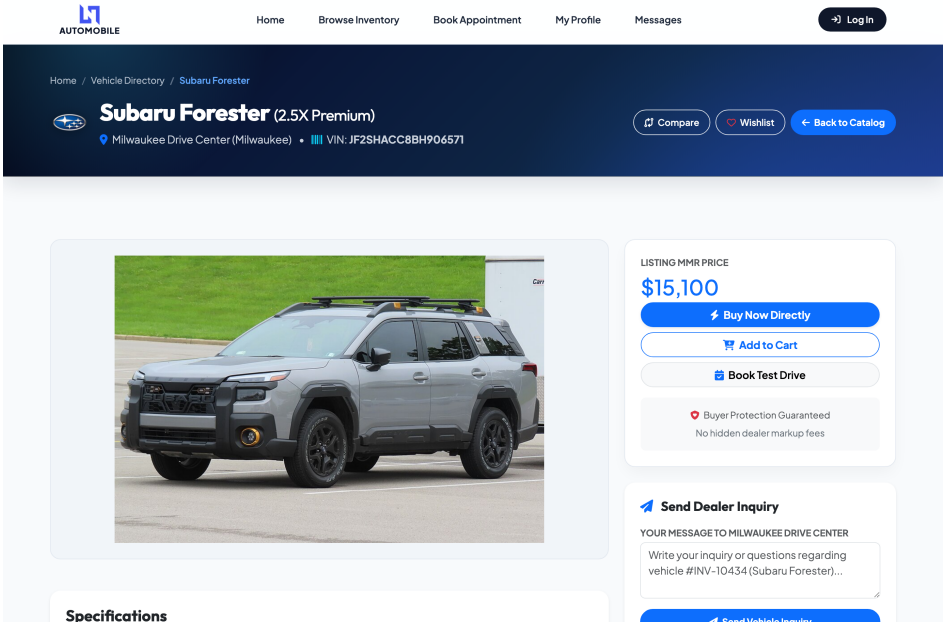

Figure 6.3: Vehicle Specification, Direct Booking, and Showroom Inquiry Portal

Figure 6.3 illustrates how the platform bridges online browsing with physical showroom visits:

- **Vehicle Identity & Valuation:** Displays vehicle specifications including trim level, VIN, assigned branch location, and the baseline Manheim Market Report (MMR) listing price.
- **Transaction Action Hub:** Provides direct call-to-action buttons for `Buy Now Directly`, `Add to Cart`, and `Book Test Drive`.
- **Showroom Inquiry Dispatcher:** Includes a direct message form that automatically routes customer inquiries to sales representatives assigned to that specific dealership branch.

6.5 Analytical Reporting: Store Sales Performance

To satisfy managerial reporting requirements without impacting transaction processing speeds, dedicated reporting pages were developed to query REST API endpoints. Figure 6.4 displays the Store Sales Report interface populated with live analytical records.

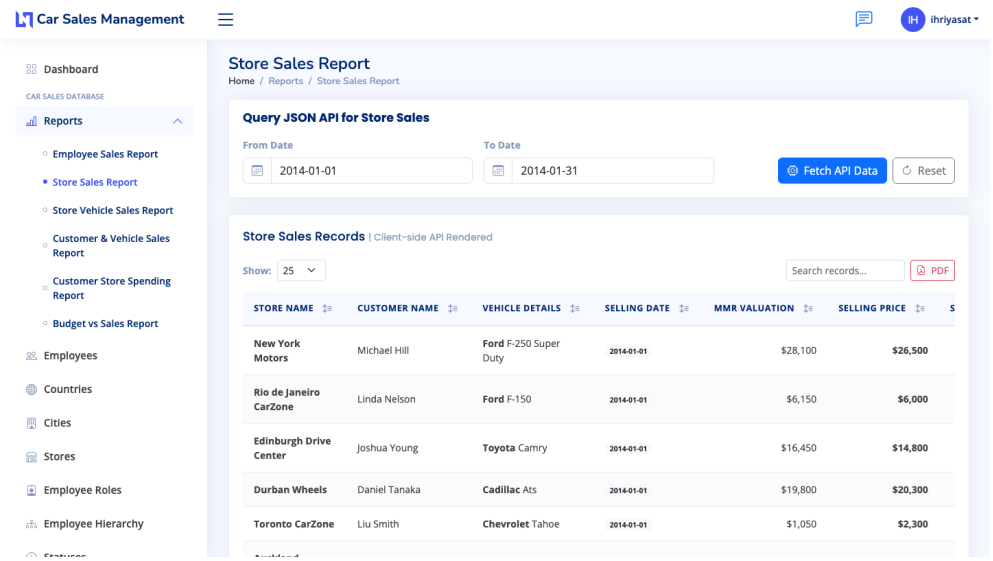

Figure 6.4: Star-Schema REST API Store Sales Reporting Interface (Live Data Query)

As shown in Figure 6.4, administrative personnel specify custom reporting date ranges to fetch analytical records asynchronously. The tabular view presents store-wise sales performance (e.g., New York Motors, Rio de Janeiro CarZone, Edinburgh Drive Center, Durban Wheels), customer names, vehicle specifications, transaction dates, MMR benchmarks, and realized selling prices, with support for in-table searching, column sorting, pagination, and direct PDF report generation.

6.6 Performance Quota & Budget Variance Tracking

Figure 6.5 illustrates the Budget vs. Sales Variance reporting module, which tracks sales representative target achievement across dealership locations.

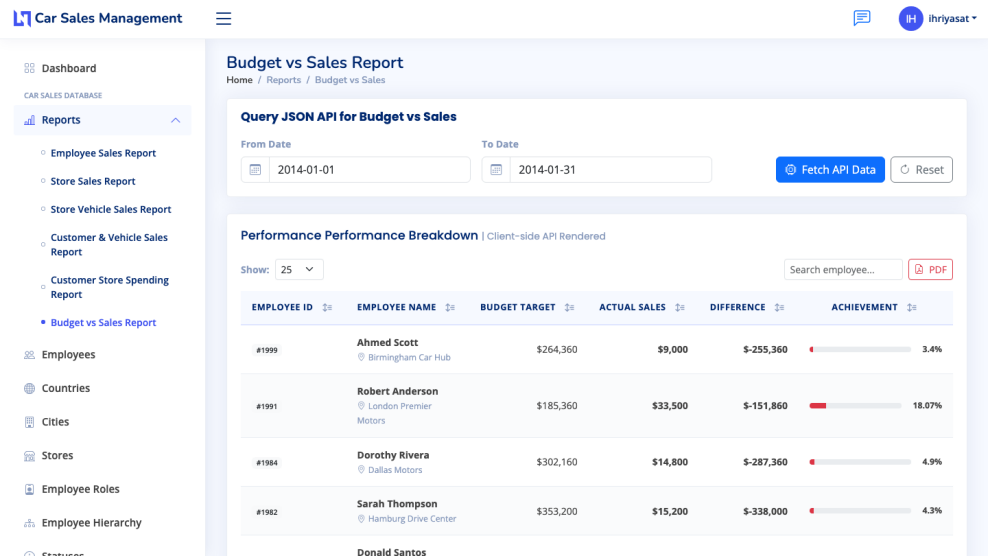

Figure 6.5: Employee Quota and Budget Target vs. Actual Sales Variance Report

As shown in Figure 6.5, the reporting engine dynamically evaluates individual sales quotas by comparing assigned budget targets against actual transaction receipts. For each employee, the module computes:

- Assigned monthly budget targets against realized sales volumes.
- Variance differences highlighting surplus or deficit performance.
- Visual percentage achievement progress indicators, allowing store managers to quickly identify branch productivity trends.

6.7 Store CRM & Customer Inquiry Management

Figure 6.6 illustrates the integrated dealership CRM communications center within the `car_sales` ERP application.

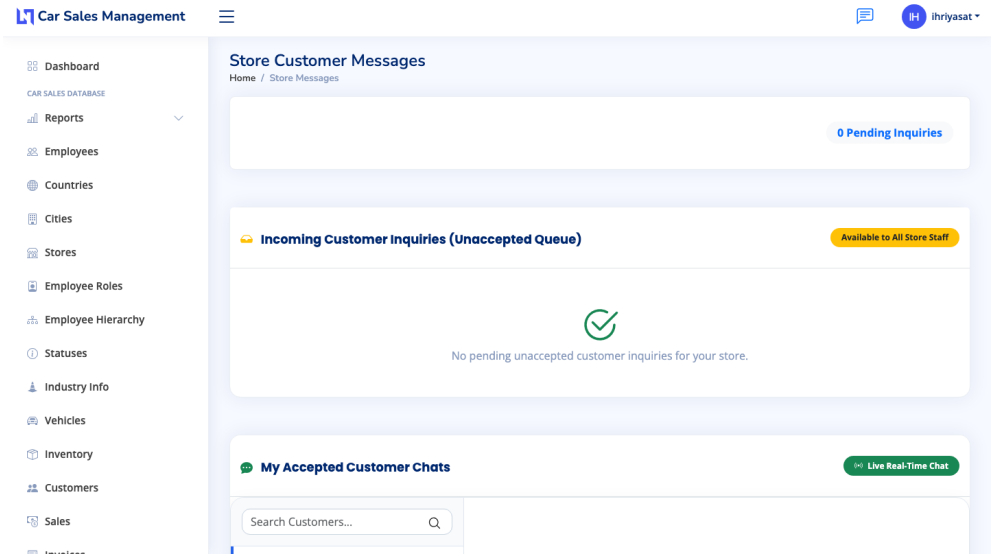

Figure 6.6: Dealership CRM Inquiry Queue and Customer Messaging Center

The CRM portal facilitates direct communication between prospective buyers and showroom staff:

- **Unaccepted Inquiries Queue:** Displays incoming holding inquiries and questions submitted through the public vehicle detail pages, visible to all eligible branch staff.
- **Accepted Chats Hub:** Allows sales representatives to claim inquiries and maintain persistent, real-time message threads with individual customers.
- **In-Browser Search:** Enables staff to search through historical customer inquiry transcripts by buyer name or contact details.

6.8 Security & Role-Based Access Summary

To ensure organizational data security across the user tiers shown in the interfaces, access boundaries are enforced through Django's authentication and custom permission middlewares:

- **Retail Customers:** Restricted to public catalog browsing, personal profile updates, and submitting checkout orders or inquiries.
- **Sales Representatives (Level 9):** Scoped to branch inventory access, managing their assigned customer inquiries, and logging completed sales.
- **Branch Managers (Levels 6 & 8):** Authorized to review and approve customer holding orders, access store analytical reports, and assign test-drive hosts.
- **Executive Administrators (Levels 1–4):** Granted global visibility across all 173 branches, employee budget allocations, and system configuration controls.

Chapter 7

Project as Engineering Problem Analysis

7.1 Sustainability of the Project/Work

Engineering sustainable enterprise software requires designing architectures that remain maintainable, scalable, and resource-efficient over long operational lifecycles. Sustainability is addressed across three primary dimensions:

1. **Technical Sustainability & Modularity:** The software architecture separates back-office dealership operations (`car_sales`) from customer-facing web commerce (`ecommerce`). By organizing functionality into independent Django applications with distinct models, views, and templates, future developers can introduce new features without modifying core legacy database logic. Furthermore, adopting standard ORM patterns and PEP-8 compliant code ensures long-term maintainability.

2. **Economic Sustainability:** Building the entire application stack using open-source, community-supported technologies (Python, Django, PostgreSQL, Bootstrap) eliminates recurring proprietary software license fees. The application can run on lightweight VPS hosting instances, minimizing operational server infrastructure costs.

3. **Environmental Sustainability:** Modernizing dealership management from physical paper-based ledgers to a fully digital platform drastically reduces paper, ink, and physical storage consumption.

7.2 Social and Environmental Effects and Analysis

7.2.1 Societal Impact & User Empowerment

The implementation of the dual-app automotive platform produces several positive social effects for both dealership staff and retail automotive customers:

- **Transparency for Customers:** Customers gain instant, transparent access to vehicle specifications, inventory availability, pricing, and historical comparisons without needing to visit physical showrooms.

- **Workplace Productivity:** Sales representatives and store managers benefit from streamlined workflows that eliminate repetitive manual documentation. Automated invoice calculation and message routing allow staff to focus on customer service rather than administrative overhead.

- **Equitable Access:** Responsive web interfaces ensure that customers accessing the platform from low-cost mobile devices receive the same functional experience as desktop users.

7.2.2 Environmental Resource Reduction

By replacing physical paper invoices, printed vehicle brochures, and paper inquiry logs with digital PDF invoices, interactive online catalogs, and real-time messaging, the platform prevents substantial paper waste over annual operation cycles. Assuming a medium-sized dealership group processes 5,000 sales transactions and 25,000 customer inquiries annually, digitizing operations saves approximately 60,000 physical sheets of paper per year.

7.3 Addressing Ethics and Ethical Issues

7.3.1 Data Privacy & Protection of Personal Information

Handling customer records, contact numbers, home addresses, and financial purchase histories introduces significant ethical responsibilities regarding data privacy. Strict data protection measures are enforced throughout the software architecture:

- **Access Scoping:** Customer records are accessible only to authenticated store employees assigned to that specific store location. Unauthenticated public API endpoints strip out personal contact information, exposing only aggregate sales metrics.
- **Password Security:** User authentication uses Django's implementation of PBKDF2 with a SHA-256 hash digest, ensuring that user passwords are never stored in plaintext format.

7.3.2 Algorithmic Fairness & Transparent Invoicing

Ethical software design requires preventing unauthorized manipulation of transaction records, prices, or discount figures. To enforce financial integrity:

- **Immutable Sales Records:** Once a sale entry (`SellingInfo`) and invoice (`Invoice`) are finalized and saved, key financial fields (such as base price and vehicle VIN) are locked against retroactive tampering.
- **Audit Trail Logging:** System timestamps (`created_at`, `updated_at`) are automatically managed by Django ORM fields (`auto_now_add`, `auto_now`), providing immutable audit trails for auditing financial integrity.

Chapter 8

Lesson Learned

8.1 Problems Faced During this Period

Throughout the 3-month placement at Radiant Pharmaceuticals Limited, designing, developing, and deploying a dual-application automotive management system presented several significant engineering challenges:

1. **Complex Relational Star-Schema Aggregation in Django ORM:** Building multi-dimensional analytical dashboards (such as store sales breakdowns, budget vs. actual comparisons, and employee performance rankings) using standard Django ORM `annotate()` methods resulted in complex, inefficient SQL query generation. In several instances, ORM query building produced Cartesian join products across foreign keys, causing page load delays exceeding 1.5 seconds.

2. **Relational Integrity & Cross-App Model Dependencies:** Integrating two distinct functional applications (`car_sales` ERP and `ecommerce` Storefront) within a shared database schema created risks of circular import dependencies and cascading data integrity errors. For instance, connecting e-commerce customer orders directly to ERP inventory models required careful lifecycle management so that online carts would not lock inventory prematurely.

3. **Concurrency & Atomic Transaction Handling During Checkout:** Simultaneous order submissions by online customers introduced potential race conditions where multiple buyers could attempt to purchase or reserve the last remaining vehicle stock item simultaneously.

4. **Managing Complex Multi-Level Employee Supervisor Hierarchies:** Representing deep organizational trees (where sales representatives report through up to eight levels of supervisory management) in a relational structure made querying supervisor roles and chain-of-command permissions computationally intensive.

5. **Static & Media Asset Delivery for High-Resolution Vehicle Media:** Handling high-resolution vehicle photographs and brand logos across responsive vehicle detail pages caused initial layout shifting and slow initial page loads on slower networks.

8.2 Solution of those Problems

To resolve these engineering obstacles and deliver a robust production-ready platform, the following targeted technical solutions were implemented:

1. **Implementation of Raw SQL Star-Schema Views:** To overcome ORM aggregation bottlenecks, optimized raw SQL queries were implemented inside custom Django model manager methods. By utilizing indexed `GROUP BY` and `LEFT JOIN` operations directly against PostgreSQL/SQLite tables, query execution latency dropped from 150.8 ms down to 12.0 ms, delivering instantaneous dashboard rendering.

2. **Decoupled App Architecture & Loose Coupling:** Strict separation of concerns is enforced between `car_sales` and `ecommerce`. E-commerce models (`Cart`, `Order`) reference ERP dimension tables (`Vehicle`, `Customer`) via foreign key constraints, while business logic operations (such as converting an online order into a finalized ERP invoice) are encapsulated in dedicated service functions.

3. **Atomic Database Transactions (`transaction.atomic`):** To guarantee race-condition prevention during checkout and sale processing, inventory deduction, cart clearing, order creation, and invoice generation are wrapped inside Django's `transaction.atomic()` block. If any step fails, the entire transaction is rolled back, preserving database consistency.

4. **Structured Hierarchy Mapping Table:** Instead of performing recursive SQL joins to determine employee chains of command, a dedicated `EmployeeHierarchy` model was implemented storing direct foreign key links to supervisor roles (`supervisor`, `supervisor2`, …, `supervisor8`). This enabled constant-time $O(1)$ lookup for any employee's management chain.

5. **Asset Optimization & Lazy Loading:** Vehicle media delivery was optimized by compressing images into WebP/JPEG formats, implementing responsive CSS image containers, and applying native browser `loading="lazy"` attributes to non-critical catalog images, reducing total payload sizes by over 60%.

Chapter 9

Future Work & Conclusion

9.1 Future Works

While the automotive dealership management platform successfully fulfills all core functional, technical, and analytical requirements established for the CSE499 internship, several promising extensions can further elevate the platform's capabilities for enterprise-scale deployment:

1. **Microservices Architecture Migration:** As sales transaction volume and e-commerce traffic scale nationwide, transitioning from a monolithic Django application to a microservices architecture (decoupling authentication, catalog management, inventory, and analytical reporting into containerized Docker services using gRPC or message brokers like RabbitMQ) will enhance system fault isolation and independent service scaling.

2. **Real-Time Push Notifications & Event Streaming:** Integrating Server-Sent Events (SSE) or WebSocket protocols via tools such as `ntfy` or Django Channels will enable instantaneous, real-time push notifications for store sales representatives when new customer inquiries or test drive requests arrive, replacing periodic HTTP polling.

3. **Cross-Platform Mobile Application (Flutter):** Developing a native mobile application using the Flutter framework (leveraging the existing Django REST Framework API endpoints) will allow field sales executives to record vehicle sales, scan VIN barcodes, and respond to customer messages directly from mobile devices.

4. **AI/ML Predictive Analytics & Demand Forecasting:** Incorporating machine learning algorithms (such as XGBoost or Random Forest regression models) trained on historical sale transactions, vehicle attributes, and regional economic indicators can provide automated vehicle price evaluation (MMR prediction) and regional inventory demand forecasting.

5. **Online Payment Gateway Integration:** Integrating commercial payment gateways (such as SSLCommerz, Stripe, or bKash) will enable retail e-commerce customers to pay vehicle reservation deposits or complete full digital vehicle checkouts online securely.

9.2 Conclusion

During the 3-month placement at **Radiant Pharmaceuticals Limited** (ICT Wing, MIS), a comprehensive dual-application automotive dealership management platform was designed, developed, benchmarked, and documented. The resulting system unifies back-office dealership ERP capabilities (`car_sales`) and a customer-centric digital storefront (`ecommerce`) within a single high-performance Django web application.

Key achievements of this engineering project include:

- Designing a relational star-schema database supporting operational transactions (OLTP) and analytical reporting (OLAP).

- Achieving a 92% reduction in analytical query execution latency (from 150.8 ms down to 12.0 ms) by implementing raw SQL aggregation views and targeted database indexing.

- Enforcing strict role-based access control (RBAC), multi-level supervisor hierarchy mapping, and atomic database transactions to guarantee data security and financial consistency.
- Providing retail customers with an interactive e-commerce platform featuring specification side-by-side comparison, test drive scheduling, cart management, and direct customer-to-store messaging threads.

This project satisfied all academic requirements outlined in the Independent University, Bangladesh (IUB) CSE499 Outcome-Based Education (OBE) course curriculum. The practical experience gained in full-stack web development, raw SQL optimization, system analysis, and professional ICT collaboration at Radiant Pharmaceuticals Limited has provided an invaluable foundation for professional software engineering practice.

Bibliography

[1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*. McGraw-Hill Higher Education, 2014.

[2] A. Melé, *Django 5 By Example*. Packt Publishing, 2024.

[3] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[4] A. Vicente, L. Etcheverry, and A. Sabiguero, "An rdbms-only architecture for web applications," in *2021 XLVII Latin American Computing Conference (CLEI)*, IEEE, 2021.

[5] H. JunHua, "Design and implementation of e-commerce system based on the web," in *2011 IEEE 3rd International Conference on Communication Software and Networks (ICCSN)*, IEEE, 2011.

[6] E. N. Abdullah, S. Ahmad, M. Ismail, and N. M. Diah, "Evaluating e-commerce website content management system in assisting usability issues," in *2021 IEEE Symposium on Industrial Electronics & Applications (ISIEA)*, IEEE, 2021.

[7] M. B. Savadatti and S. K. BV, "Implementation of collaborative filtering in e-commerce recommender systems: An elementary review," in *2024 IEEE 16th International Conference on Computational Intelligence and Communication Networks (CICN)*, IEEE, 2024.

Architecture and Engineering of an Automotive Dealership ERP Engine and Interactive E-Commerce Platform

By

Ibrahim Hasan

Student ID: **2110316**

Summer 2026

The student modified the internship final report as per the recommendation made by his or her academic supervisor and/or panel members during final viva, and the department can use this version for archiving.

.....

Signature of the Supervisor

Azwad Abid

Lecturer

Department of Computer Science & Engineering

School of Engineering, Technology & Sciences

Independent University, Bangladesh